Full-Stack Developer Revision Guide

Basics to Advanced — with coding examples

JavaScript (ES2024+) • TypeScript

React (18 / 19) • Node.js & Express

Python • Django & Django REST Framework

SQL (PostgreSQL / MySQL)

Table of Contents

Part 1 — JavaScript: Basics to Advanced

- 1.1 How JavaScript Runs
- 1.2 Variables, Scope and Hoisting
- 1.3 Data Types
- 1.4 Functions
- 1.5 this Binding Rules
- 1.6 Closures
- 1.7 Objects and Prototypes
- 1.8 Classes (ES6+)
- 1.9 Arrays — Essential Methods
- 1.10 Modern Syntax (ES6 → ES2024)
- 1.11 Map, Set, WeakMap, WeakSet
- 1.12 Iterators and Generators
- 1.13 The Event Loop
- 1.14 Promises and Async/Await
- 1.15 Error Handling
- 1.16 Modules
- 1.17 Memory Management and Performance
- 1.18 Browser and DOM Essentials
- 1.19 Functional Programming Concepts
- 1.20 Design Patterns in JavaScript
- 1.21 Deep Clone and Deep Equal (Interview Implementations)
- 1.22 Quick Revision — Tricky Output Questions

Part 2 — TypeScript: Basics to Advanced

- 2.1 Why TypeScript
- 2.2 Key tsconfig.json Options
- 2.3 Basic Types
- 2.4 Interfaces vs Type Aliases
- 2.5 Functions
- 2.6 Narrowing and Type Guards
- 2.7 Generics
- 2.8 Built-in Utility Types
- 2.9 Advanced Types
- 2.10 Classes in TypeScript
- 2.11 Enums vs Union Literals
- 2.12 Decorators (TS 5 Standard Decorators)
- 2.13 Declaration Files and Module Augmentation
- 2.14 TypeScript with React
- 2.15 TypeScript with Node/Express
- 2.16 Type Challenges (Practice)
- 2.17 Variance, any vs unknown vs never — Summary

Part 3 — React: Basics to Advanced

- 3.1 Core Ideas
- 3.2 Components and Props
- 3.3 Hooks — Complete Reference
- 3.4 Custom Hooks

3.5 Stale Closures — The Most Common Hook Bug
3.6 Performance Optimization
3.7 Advanced Component Patterns
3.8 Error Boundaries
3.9 Class Component Lifecycle (for legacy code)
3.10 State Management
3.11 Routing (React Router v6/v7)
3.12 Forms — React Hook Form + Zod
3.13 Rendering Strategies and Server Components
3.14 Testing React
3.15 Interview Questions Checklist

Part 4 — Node.js: Basics to Advanced

4.1 Architecture
4.2 The Node.js Event Loop Phases
4.3 Modules
4.4 Core Modules
4.5 EventEmitter
4.6 Buffers and Streams
4.7 HTTP Server from Scratch
4.8 Express.js In Depth
4.9 Authentication and Authorization
4.10 Databases from Node
4.11 Caching with Redis
4.12 Scaling Node.js
4.13 Background Jobs and Queues
4.14 Real-time: WebSockets / Socket.IO
4.15 Error Handling and Process Management
4.16 Configuration and Logging
4.17 Security Checklist
4.18 Performance
4.19 Testing Node
4.20 REST API Design Quick Reference

Part 5 — Python: Basics to Advanced

5.1 How Python Runs
5.2 Core Data Types
5.3 Control Flow and Comprehensions
5.4 Functions
5.5 Decorators
5.6 Iterators and Generators
5.7 Object-Oriented Python
5.8 Context Managers
5.9 Exceptions
5.10 Type Hints
5.11 Concurrency: Threads, Processes, Asyncio
5.12 Modules, Packages and Environments
5.13 Memory Model and Internals
5.14 Metaclasses (Advanced)
5.15 Testing with pytest
5.16 Common Interview Snippets

Part 6 — Django & Django REST Framework

6.1 Architecture: MTV and the Request Lifecycle
6.2 Settings Essentials
6.3 Models
6.4 ORM — QuerySets
6.5 Transactions and Concurrency
6.6 Migrations
6.7 Views
6.8 Templates and Forms (Quick)
6.9 Middleware
6.10 Signals
6.11 Authentication and Permissions
6.12 Django REST Framework (DRF)
6.13 Caching
6.14 Celery (Background Tasks)
6.15 Async Django
6.16 Testing Django
6.17 Security Checklist
6.18 Performance and Deployment
6.19 Service Layer Pattern (Clean Django)
6.20 Interview Questions Checklist

Part 7 — SQL: Basics to Advanced

7.1 SQL Sub-languages
7.2 Data Types (Common)
7.3 DDL and Constraints
7.4 Basic DML and Queries
7.5 Logical Order of Execution
7.6 Aggregation: GROUP BY and HAVING
7.7 Joins
7.8 Set Operations
7.9 Subqueries
7.10 Common Table Expressions (CTEs)
7.11 Window Functions
7.12 Classic SQL Interview Problems
7.13 Indexes
7.14 Query Optimization and EXPLAIN
7.15 Transactions and ACID
7.16 Normalization
7.17 Views, Materialized Views, Functions, Triggers
7.18 Working with JSON, Strings and Dates (PostgreSQL)
7.19 Security, Roles and SQL Injection
7.20 Scaling Databases
7.21 Quick Reference — Postgres vs MySQL

Part 1 — JavaScript: Basics to Advanced

1.1 How JavaScript Runs

JavaScript is a single-threaded, dynamically typed, prototype-based language with first-class functions. It runs inside a **host environment** (browser, Node.js, Deno, Bun) that provides APIs such as the DOM, `fetch`, timers, and file system access.

Engine pipeline (V8): source → parser → AST → Ignition (bytecode interpreter) → TurboFan (optimizing JIT compiler). Hot functions are optimized based on observed types; if types change ("megamorphic" call sites), code is deoptimized. Practical takeaway: keep object shapes consistent (initialize all properties in the constructor, in the same order).

Execution context has three parts:

1. **Variable environment** — `var` declarations and function declarations.
2. **Lexical environment** — `let`, `const`, `class` bindings, plus a reference to the outer environment (this chain is the *scope chain*).
3. `this` **binding**.

Each context is created in two phases: **creation** (hoisting — memory allocated for declarations) and **execution** (code runs line by line).

1.2 Variables, Scope and Hoisting

Keyword	Scope	Hoisted?	Re-declare	Re-assign	TDZ
`var`	Function	Yes, initialized to `undefined`	Yes	Yes	No
`let`	Block	Yes, uninitialized	No	Yes	Yes
`const`	Block	Yes, uninitialized	No	No (binding only)	Yes

```
console.log(a); // undefined   (var hoisted & initialized)
var a = 1;

console.log(b); // ReferenceError: Cannot access 'b' before initialization (TDZ)
let b = 2;

const obj = { x: 1 };
obj.x = 2;      // OK — const freezes the binding, not the value
// obj = {};    // TypeError

// Function declarations are fully hoisted; function expressions are not
hoisted();      // works
function hoisted() { return 'hi'; }

notHoisted();   // TypeError: notHoisted is not a function
var notHoisted = function () {};
```

Classic loop bug — `var` is function-scoped, so all callbacks share one `i`:

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j)); // 0 1 2 (new binding per
iteration)
```

1.3 Data Types

Primitives (7): `string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, `null`. Primitives are immutable and compared by value. **Reference type:** `object` (includes arrays, functions, dates, maps, sets...). Compared by reference.

```
typeof null;          // 'object'  (historical bug)
typeof function(){};  // 'function'
typeof 10n;           // 'bigint'
Array.isArray([]);    // true — the reliable array check

0.1 + 0.2 === 0.3;                    // false (IEEE-754 double)
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON; // true
Number.MAX_SAFE_INTEGER;              // 9007199254740991 → use BigInt beyond
NaN === NaN;                          // false → use Number.isNaN(x)
Object.is(NaN, NaN);                  // true
Object.is(0, -0);                     // false
```

Type Coercion and Equality

`==` performs coercion; `===` does not. Always prefer `===` except `x == null` (checks both `null` and `undefined`).

```
'5' + 3      // '53'  (+ with a string concatenates)
'5' - 3      // 2     (- coerces to number)
[] + {}      // '[object Object]'
[] == false  // true  ([] → '' → 0, false → 0)
null == 0    // false (null only == undefined)
!!''         // false

// Falsy values: false, 0, -0, 0n, '', null, undefined, NaN — everything else is truthy
```

Copying: Shallow vs Deep

```
const original = { a: 1, nested: { b: 2 } };

const shallow = { ...original };          // or Object.assign({}, original)
shallow.nested.b = 99;                    // also changes original.nested.b!

const deep = structuredClone(original);   // modern deep clone (handles Map, Set, Date, cycles)
// JSON.parse(JSON.stringify(o)) loses functions, undefined, Dates, Maps
```

1.4 Functions

```
// Declaration
function add(a, b = 0) { return a + b; }    // default parameter

// Expression & arrow
const mul = (a, b) => a * b;

// Rest & spread
function sum(...nums) { return nums.reduce((acc, n) => acc + n, 0); }
```

```
sum(...[1, 2, 3]); // 6

// IIFE
(function () { /* private scope */ })();

// Higher-order function
const withLogging = (fn) => (...args) => {
  console.log('calling with', args);
  return fn(...args);
};
```

Arrow functions vs regular functions:

- No own `this` — they capture `this` lexically from the enclosing scope.
- No `arguments` object, cannot be used with `new`, no `prototype`.
- Ideal for callbacks; avoid as object methods that need `this`.

1.5 `this` Binding Rules

Precedence (highest first):

1. **`new` binding** — `new Foo()` → `this` is the new object.
2. **Explicit** — `call`, `apply`, `bind`.
3. **Implicit** — `obj.method()` → `this` is `obj`.
4. **Default** — standalone call → `undefined` in strict mode, `globalThis` otherwise.
5. Arrow functions ignore all rules and use lexical `this`.

```
const user = {
  name: 'Anish',
  greet() { return `Hi ${this.name}`; },
  greetLater() { setTimeout(() => console.log(this.name), 0); }, // arrow keeps this
};

const g = user.greet;
g();                      // 'Hi undefined' — lost implicit binding
g.call({ name: 'Ravi' });   // 'Hi Ravi'
g.apply({ name: 'Ravi' }, []);
const bound = g.bind(user); // permanently bound
bound();                  // 'Hi Anish'
```

Polyfill for `bind` (common interview question)

```
Function.prototype.myBind = function (ctx, ...preset) {
  const fn = this;
  return function (...later) {
    return fn.apply(ctx, [...preset, ...later]);
  };
};
```

1.6 Closures

A **closure** is a function bundled with references to its surrounding lexical environment. The inner function keeps access to outer variables even after the outer function returns.

```
function createCounter() {
  let count = 0;                // private state
```

```
  return {
    inc: () => ++count,
    dec: () => --count,
    get: () => count,
  };
}
const c = createCounter();
c.inc(); c.inc();
c.get(); // 2

// Memoization with closures
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));
    return cache.get(key);
  };
}

// once()
function once(fn) {
  let called = false, result;
  return (...args) => (called ? result : (called = true, result = fn(...args)));
}
```

Uses: data privacy, factories, memoization, currying, debouncing, React hooks (each render closes over its props/state — the root of the "stale closure" bug).

Currying and Partial Application

```
const curry = (fn) =>
  function curried(...args) {
    return args.length >= fn.length
      ? fn.apply(this, args)
      : (...next) => curried.apply(this, [...args, ...next]);
  };

const add3 = curry((a, b, c) => a + b + c);
add3(1)(2)(3); // 6
add3(1, 2)(3); // 6
```

Debounce and Throttle

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

function throttle(fn, interval) {
  let last = 0;
  return function (...args) {
    const now = Date.now();
    if (now - last >= interval) {
      last = now;
      fn.apply(this, args);
    }
  };
}
// debounce → search box (fire after the user stops typing)
// throttle → scroll/resize handlers (fire at most every N ms)
```

1.7 Objects and Prototypes

Every object has an internal `[[Prototype]]` link. Property lookup walks the **prototype chain** until `null`.

```
const animal = { eats: true, walk() { return 'walking'; } };
const dog = Object.create(animal);   // dog.[[Prototype]] = animal
dog.barks = true;
dog.walk();                          // 'walking' (found on prototype)
Object.getPrototypeOf(dog) === animal; // true
dog.hasOwnProperty('eats');          // false
Object.hasOwn(dog, 'barks');         // true (modern)

// Constructor function (pre-ES6 pattern)
function Person(name) { this.name = name; }
Person.prototype.hello = function () { return `I am ${this.name}`; };
const p = new Person('A');
p.__proto__ === Person.prototype;    // true
```

What `new` does: (1) creates an empty object, (2) links its prototype to `Constructor.prototype`, (3) calls the constructor with `this` = new object, (4) returns the object (unless the constructor returns another object).

Object utilities

```
const o = { a: 1, b: 2 };
Object.keys(o);      // ['a','b']
Object.values(o);    // [1,2]
Object.entries(o);   // [['a',1],['b',2]]
Object.fromEntries([['x', 1]]); // { x: 1 }

Object.freeze(o);    // shallow: no add/remove/change
Object.seal(o);      // no add/remove, can change existing

Object.defineProperty(o, 'secret', {
  value: 42, writable: false, enumerable: false, configurable: false,
});

// Getters / setters
const temp = {
  _c: 0,
  get f() { return this._c * 9 / 5 + 32; },
  set f(v) { this._c = (v - 32) * 5 / 9; },
};
```

1.8 Classes (ES6+)

Classes are syntactic sugar over prototypes, with some extras (private fields, static blocks).

```
class Account {
  static #count = 0;           // private static
  #balance = 0;                // private instance field

  constructor(owner) {
    this.owner = owner;
    Account.#count++;
  }

  deposit(amount) {
    if (amount <= 0) throw new RangeError('Amount must be positive');
```

```
    this.#balance += amount;
    return this;               // enables chaining
  }

  get balance() { return this.#balance; }

  static get count() { return Account.#count; }
}

class SavingsAccount extends Account {
  constructor(owner, rate) {
    super(owner);              // must call before using `this`
    this.rate = rate;
  }
  addInterest() {
    return this.deposit(this.balance * this.rate);
  }
}

const s = new SavingsAccount('Anish', 0.05).deposit(1000).addInterest();
s.balance;         // 1050
Account.count;     // 1
// s.#balance      // SyntaxError — truly private
```

1.9 Arrays — Essential Methods

```
const users = [
  { id: 1, name: 'A', age: 30, role: 'admin' },
  { id: 2, name: 'B', age: 22, role: 'user' },
  { id: 3, name: 'C', age: 35, role: 'user' },
];

users.map(u => u.name);                    // ['A','B','C']
users.filter(u => u.age > 25);             // A, C
users.find(u => u.id === 2);               // B
users.findIndex(u => u.id === 2);          // 1
users.findLast(u => u.role === 'user');    // C (ES2023)
users.some(u => u.role === 'admin');       // true
users.every(u => u.age > 18);              // true
users.reduce((sum, u) => sum + u.age, 0); // 87

// Group by (ES2024)
Object.groupBy(users, u => u.role); // { admin: [A], user: [B, C] }

// Group by with reduce (works everywhere)
const byRole = users.reduce((acc, u) => {
  (acc[u.role] ??= []).push(u);
  return acc;
}, {});

// Non-mutating versions (ES2023)
const sorted = users.toSorted((a, b) => a.age - b.age);
const reversed = users.toReversed();
const replaced = users.with(0, { ...users[0], name: 'Z' });

// Mutating: push, pop, shift, unshift, splice, sort, reverse, fill
[3, 1, 10].sort();          // [1, 10, 3]  — default sort is lexicographic!
[3, 1, 10].sort((a, b) => a - b); // [1, 3, 10]

[[1, [2]], [3]].flat(Infinity);   // [1, 2, 3]
['a b', 'c'].flatMap(s => s.split(' ')); // ['a','b','c']
Array.from({ length: 3 }, (_, i) => i * 2); // [0,2,4]
[...new Set([1, 1, 2])];          // [1, 2] — dedupe
[1, 2, 3].at(-1);                 // 3
```

Implement `map` and `reduce` polyfills

```
Array.prototype.myMap = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  }
  return out;
};

Array.prototype.myReduce = function (cb, init) {
  let i = 0, acc = init;
  if (arguments.length < 2) {
    if (!this.length) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[0]; i = 1;
  }
  for (; i < this.length; i++) acc = cb(acc, this[i], i, this);
  return acc;
};
```

1.10 Modern Syntax (ES6 → ES2024)

```
// Destructuring with defaults, renaming, rest
const { name: userName = 'guest', ...rest } = { name: 'A', age: 1, role: 'x' };
const [first, , third = 0] = [1, 2];

// Optional chaining & nullish coalescing
const city = user?.address?.city ?? 'Unknown';  // ?? only falls back for null/undefined
const count = 0 || 10;   // 10
const count2 = 0 ?? 10;  // 0

// Logical assignment
opts.timeout ??= 3000;
opts.retries ||= 3;
opts.enabled &&= isAllowed;

// Template literals & tagged templates
const tag = (strings, ...vals) => strings.raw.join('|') + vals.join(',');
tag`a${1}b${2}`;

// Computed keys & shorthand
const key = 'dynamic';
const o2 = { [key]: 1, userName };

// Numeric separators, exponent
const big = 1_000_000;  2 ** 10;

// Symbol — unique keys, well-known symbols
const id = Symbol('id');
const objWithSym = { [id]: 123 };
```

1.11 Map, Set, WeakMap, WeakSet

```
const m = new Map([['a', 1]]);
m.set({ k: 1 }, 'object key OK');  // any key type, keeps insertion order
m.size;

const s = new Set([1, 2, 2]);       // unique values
s.has(2);
// ES2025 Set methods: union, intersection, difference, isSubsetOf...
new Set([1, 2]).union(new Set([2, 3])); // Set {1,2,3}
```

```
// WeakMap: keys must be objects, held weakly (garbage-collectable), not iterable
const privateData = new WeakMap();
class Thing {
  constructor() { privateData.set(this, { secret: 1 }); }
}
```

When to use Map over Object: frequent add/remove, non-string keys, need size/order, avoiding prototype key collisions.

1.12 Iterators and Generators

```
// Custom iterable
const range = {
  from: 1, to: 4,
  [Symbol.iterator]() {
    let cur = this.from, last = this.to;
    return { next: () => (cur <= last ? { value: cur++, done: false } : { value: undefined,
done: true }) };
  },
};
[...range]; // [1,2,3,4]

// Generator — pausable function
function* idGen() {
  let id = 0;
  while (true) {
    const reset = yield ++id;
    if (reset) id = 0;
  }
}
const gen = idGen();
gen.next().value;      // 1
gen.next().value;      // 2
gen.next(true).value;  // 1

// Async generator — paginate an API
async function* fetchPages(url) {
  let next = url;
  while (next) {
    const res = await fetch(next).then(r => r.json());
    yield res.items;
    next = res.nextPage;
  }
}
for await (const items of fetchPages('/api/items?page=1')) console.log(items);
```

1.13 The Event Loop

JavaScript has one call stack. Async work is delegated to the host (Web APIs / libuv); completed callbacks are queued.

- **Call stack** — synchronous code.
- **Microtask queue** — Promise callbacks (`then/catch/finally`), `queueMicrotask`, `MutationObserver`, `await` continuations.
- **Macrotask (task) queue** — `setTimeout`, `setInterval`, I/O, UI events, `MessageChannel`.

Algorithm: run one macrotask → drain *all* microtasks → render (browser) → next macrotask.

```
console.log('1 sync');
setTimeout(() => console.log('2 timeout'), 0);
```

```
Promise.resolve().then(() => console.log('3 micro'));
queueMicrotask(() => console.log('4 micro'));
(async () => {
  console.log('5 async start (sync)');
  await null;
  console.log('6 after await (micro)');
})();
console.log('7 sync');

// Output: 1, 5, 7, 3, 4, 6, 2
```

1.14 Promises and Async/Await

A Promise is in one of three states: **pending**, **fulfilled**, **rejected**. Once settled it is immutable.

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

function getUser(id) {
  return new Promise((resolve, reject) => {
    if (!id) return reject(new Error('id required'));
    setTimeout(() => resolve({ id, name: 'A' }), 100);
  });
}

getUser(1)
  .then(u => u.name)
  .then(console.log)
  .catch(console.error)
  .finally(() => console.log('done'));

// Combinators
await Promise.all([p1, p2]);        // all fulfill, or reject on first rejection
await Promise.allSettled([p1, p2]); // [{status:'fulfilled',value}, {status:'rejected',reason}]
await Promise.race([p1, timeout]);  // first to settle
await Promise.any([p1, p2]);        // first to fulfill; AggregateError if all reject

// Promise.withResolvers (ES2024)
const { promise, resolve, reject } = Promise.withResolvers();
```

async/await patterns

```
// Sequential (slow) vs parallel (fast)
async function sequential() {
  const a = await fetchA();  // waits
  const b = await fetchB();  // then waits
  return [a, b];
}
async function parallel() {
  const [a, b] = await Promise.all([fetchA(), fetchB()]);
  return [a, b];
}

// Error handling
async function load() {
  try {
    const res = await fetch('/api');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error(err);
    throw err;
  }
}

// Timeout + abort
```

```
async function fetchWithTimeout(url, ms = 5000) {
  const controller = new AbortController();
  const id = setTimeout(() => controller.abort(), ms);
  try {
    return await fetch(url, { signal: controller.signal });
  } finally {
    clearTimeout(id);
  }
}
// or: fetch(url, { signal: AbortSignal.timeout(5000) })

// Retry with exponential backoff
async function retry(fn, retries = 3, delay = 500) {
  try {
    return await fn();
  } catch (e) {
    if (retries === 0) throw e;
    await wait(delay);
    return retry(fn, retries - 1, delay * 2);
  }
}

// Concurrency limit (run N at a time)
async function mapLimit(items, limit, fn) {
  const results = [];
  let i = 0;
  const workers = Array.from({ length: limit }, async () => {
    while (i < items.length) {
      const idx = i++;
      results[idx] = await fn(items[idx]);
    }
  });
  await Promise.all(workers);
  return results;
}
```

Implement `Promise.all` (interview favourite)

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let done = 0;
    if (promises.length === 0) return resolve(results);
    promises.forEach((p, i) => {
      Promise.resolve(p).then((val) => {
        results[i] = val;
        if (++done === promises.length) resolve(results);
      }, reject);
    });
  });
}
```

1.15 Error Handling

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

try {
  throw new ValidationError('email', 'Invalid email');
} catch (e) {
```

```
  if (e instanceof ValidationError) console.log(e.field, e.message);
  else throw e;
} finally {
  // always runs
}

// Error cause (ES2022)
throw new Error('Failed to save user', { cause: originalErr });

// Global handlers
window.addEventListener('unhandledrejection', (e) => console.error(e.reason));
```

1.16 Modules

```
// ES Modules (static, analyzable, tree-shakable, async loading)
export const PI = 3.14;
export default function area(r) { return PI * r * r; }
import area, { PI } from './math.js';
import * as math from './math.js';

// Dynamic import → code splitting
const { default: chart } = await import('./chart.js');

// CommonJS (Node legacy: synchronous, dynamic)
const fs = require('fs');
module.exports = { helper };
```

ESM	CommonJS
`import/export`	`require/module.exports`
Static, hoisted, tree-shakable	Dynamic, runtime
Live bindings	Copies of exported values
Top-level `await` allowed	No top-level await
`this` is `undefined` at top level	`this` is `module.exports`

1.17 Memory Management and Performance

- Garbage collection uses **mark-and-sweep** (generational: young/old space in V8).
- **Common leaks:** forgotten timers/intervals, detached DOM nodes, global variables, closures holding large objects, event listeners never removed, unbounded caches (use `WeakMap`/LRU).
- Use `WeakRef` and `FinalizationRegistry` only for advanced caching scenarios.

```
// Leak: listener never removed
function attach() {
  const big = new Array(1e6).fill('*');
  window.addEventListener('resize', () => console.log(big.length));
}
// Fix: keep a reference and removeEventListener, or use AbortController
const ac = new AbortController();
window.addEventListener('resize', handler, { signal: ac.signal });
ac.abort(); // removes listener
```

1.18 Browser and DOM Essentials

```
// Event delegation — one listener for many children
document.querySelector('#list').addEventListener('click', (e) => {
  const item = e.target.closest('li[data-id]');
  if (item) console.log('clicked', item.dataset.id);
});

// Phases: capture (top→down) → target → bubble (bottom→up)
el.addEventListener('click', fn, { capture: true, once: true, passive: true });
e.stopPropagation();  // stop bubbling
e.preventDefault();   // stop default action (form submit, link nav)

// Intersection Observer — lazy loading / infinite scroll
const io = new IntersectionObserver((entries) => {
  entries.forEach((en) => en.isIntersecting && loadMore());
});
io.observe(document.querySelector('#sentinel'));
```

Storage: `localStorage` (persistent, ~5MB, sync, strings only), `sessionStorage` (per tab), cookies (sent with requests; use `HttpOnly`, `Secure`, `SameSite`), IndexedDB (large structured async storage).

Rendering: DOM + CSSOM → render tree → layout (reflow) → paint → composite. Avoid layout thrashing (reading `offsetHeight` after writes in a loop); animate `transform` / `opacity`.

Security: XSS (never inject untrusted HTML; use `textContent`; CSP), CSRF (SameSite cookies, CSRF tokens), CORS (server-side header controlling cross-origin reads).

1.19 Functional Programming Concepts

- **Pure functions** — same input → same output, no side effects.
- **Immutability** — return new objects instead of mutating.
- **Composition** — build complex functions from small ones.

```
const compose = (...fns) => (x) => fns.reduceRight((v, f) => f(v), x);
const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);

const slugify = pipe(
  (s) => s.trim(),
  (s) => s.toLowerCase(),
  (s) => s.replace(/\s+/g, '-'),
);
slugify('  Hello World '); // 'hello-world'
```

1.20 Design Patterns in JavaScript

```
// Singleton (module scope is a natural singleton)
let instance;
export function getDb() { return (instance ??= createConnection()); }

// Observer / Pub-Sub — basis of EventEmitter, Redux subscribe
class EventBus {
  #events = new Map();
  on(evt, fn) {
    if (!this.#events.has(evt)) this.#events.set(evt, new Set());
    this.#events.get(evt).add(fn);
```

```
    return () => this.#events.get(evt).delete(fn); // unsubscribe
  }
  emit(evt, ...args) { this.#events.get(evt)?.forEach((fn) => fn(...args)); }
}

// Factory
function createShape(type) {
  const shapes = { circle: () => new Circle(), square: () => new Square() };
  return shapes[type]?.() ?? (() => { throw new Error('Unknown shape'); })();
}

// Proxy — validation, reactivity (Vue 3 uses this)
const validated = new Proxy({}, {
  set(target, prop, value) {
    if (prop === 'age' && !Number.isInteger(value)) throw new TypeError('age must be int');
    target[prop] = value;
    return true;
  },
});
```

1.21 Deep Clone and Deep Equal (Interview Implementations)

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;
  if (seen.has(value)) return seen.get(value);       // handle cycles
  if (value instanceof Date) return new Date(value);
  if (value instanceof Map) return new Map([...value].map(([k, v]) => [k, deepClone(v, seen)]));
  if (value instanceof Set) return new Set([...value].map(v => deepClone(v, seen)));
  const out = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, out);
  for (const key of Reflect.ownKeys(value)) out[key] = deepClone(value[key], seen);
  return out;
}

function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || !a || !b) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  if (ka.length !== kb.length) return false;
  return ka.every((k) => deepEqual(a[k], b[k]));
}

// Flatten an object: { a: { b: 1 } } → { 'a.b': 1 }
function flattenObj(obj, prefix = '', out = {}) {
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === 'object' && !Array.isArray(v)) flattenObj(v, key, out);
    else out[key] = v;
  }
  return out;
}
```

1.22 Quick Revision — Tricky Output Questions

```
// 1
console.log([1, 2, 3] + [4]);       // '1,2,34'
// 2
console.log(typeof typeof 1);       // 'string'
// 3
const x = { a: 1 };
const y = x; y.a = 2;
console.log(x.a);                   // 2
```

```
// 4
console.log([10, 1, 3].sort());     // [1, 10, 3]
// 5
let n = 1;
{ let n = 2; }
console.log(n);                     // 1
// 6
console.log(1 < 2 < 3, 3 > 2 > 1);  // true false   (true > 1 → 1 > 1)
// 7
const fn = () => arguments;         // ReferenceError in modules (no own arguments)
// 8
console.log(+'', +' ', +'a');       // 0 0 NaN
// 9
for (const k in [7, 8]) console.log(typeof k); // 'string' 'string' — for...in gives keys
```

Part 2 — TypeScript: Basics to Advanced

2.1 Why TypeScript

TypeScript is a **statically typed superset** of JavaScript that compiles (transpiles) to plain JS. Types are erased at runtime — they exist only for the compiler and editor. Benefits: early error detection, refactoring safety, self-documenting APIs, excellent IntelliSense.

Structural typing ("duck typing"): compatibility is based on shape, not on declared names.

```
interface Point { x: number; y: number }
const p = { x: 1, y: 2, z: 3 };
const q: Point = p;            // OK — p has at least x and y
// const r: Point = { x: 1, y: 2, z: 3 }; // Error: excess property check on literals
```

2.2 Key `tsconfig.json` Options

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,                  // enables all strict checks below
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUncheckedIndexedAccess": true, // arr[i] is T | undefined
    "exactOptionalPropertyTypes": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist",
    "rootDir": "src",
    "declaration": true,
    "sourceMap": true,
    "jsx": "react-jsx",
    "paths": { "@/*": ["./src/*"] }
  }
}
```

2.3 Basic Types

```
let id: number = 1;
let name: string = 'Anish';
let active: boolean = true;
let big: bigint = 100n;
let nothing: null = null;
let notSet: undefined = undefined;

let list: number[] = [1, 2];
let list2: Array<string> = ['a'];
let tuple: [string, number] = ['age', 30];
let namedTuple: [name: string, age?: number] = ['A'];
let readonlyArr: readonly number[] = [1, 2];
```

```ts
let anything: any = 4;       // opts out of type checking — avoid
let safe: unknown = 4;       // must narrow before use — prefer over any
if (typeof safe === 'number') safe.toFixed(2);

function fail(msg: string): never { throw new Error(msg); } // never returns
function log(msg: string): void { console.log(msg); }

// Literal types
let dir: 'up' | 'down' = 'up';
const PORT = 3000;           // type is literal 3000
```

2.4 Interfaces vs Type Aliases

```ts
interface User {
  readonly id: number;
  name: string;
  email?: string;                  // optional
  [key: string]: unknown;          // index signature
}

interface Admin extends User { permissions: string[] }

// Declaration merging — only interfaces
interface Window { myGlobal: string }

type ID = string | number;         // unions — only type aliases
type Point = { x: number; y: number };
type Point3D = Point & { z: number }; // intersection
type Handler = (e: Event) => void;
```

	`interface`	`type`
Object shapes	Yes	Yes
Unions / primitives / tuples	No	Yes
Declaration merging	Yes	No
`extends`	Yes	Via `&`
Mapped/conditional types	No	Yes

Rule of thumb: `interface` for public object contracts and class shapes; `type` for unions, utility compositions and everything else.

2.5 Functions

```ts
function add(a: number, b: number = 0): number { return a + b; }
const greet = (name: string, greeting?: string): string => `${greeting ?? 'Hi'} ${name}`;
function sum(...nums: number[]): number { return nums.reduce((a, b) => a + b, 0); }

// Function type
type BinaryOp = (a: number, b: number) => number;

// Overloads
function parse(input: string): number;
function parse(input: number): string;
function parse(input: string | number) {
  return typeof input === 'string' ? Number(input) : String(input);
}
```

```ts
// `this` parameter
function onClick(this: HTMLButtonElement, e: MouseEvent) { this.disabled = true; }
```

2.6 Narrowing and Type Guards

```ts
function format(v: string | number | Date | null) {
  if (v == null) return '';                 // null/undefined narrowing
  if (typeof v === 'string') return v.toUpperCase();
  if (typeof v === 'number') return v.toFixed(2);
  if (v instanceof Date) return v.toISOString();
}

// `in` narrowing
type Fish = { swim: () => void };
type Bird = { fly: () => void };
function move(a: Fish | Bird) { 'swim' in a ? a.swim() : a.fly(); }

// User-defined type guard
function isString(x: unknown): x is string { return typeof x === 'string'; }

// Assertion function
function assertDefined<T>(v: T, msg = 'missing'): asserts v is NonNullable<T> {
  if (v == null) throw new Error(msg);
}
```

Discriminated Unions + Exhaustiveness

```ts
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rect'; w: number; h: number }
  | { kind: 'triangle'; base: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle':   return Math.PI * s.radius ** 2;
    case 'rect':     return s.w * s.h;
    case 'triangle': return 0.5 * s.base * s.height;
    default: {
      const _exhaustive: never = s; // compile error if a case is missing
      return _exhaustive;
    }
  }
}

// Typical API state modelling
type AsyncState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };
```

2.7 Generics

```ts
function identity<T>(value: T): T { return value; }
identity<string>('a');
identity(42);                       // inferred T = number

// Constraints
function getLength<T extends { length: number }>(x: T) { return x.length; }

// keyof constraint — type-safe property access
```

```ts
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] { return obj[key]; }
getProp({ a: 1, b: 'x' }, 'b');    // string
// getProp({ a: 1 }, 'z');         // Error

// Generic interfaces & defaults
interface ApiResponse<T = unknown> {
  data: T;
  error?: string;
  status: number;
}

// Generic class
class Stack<T> {
  private items: T[] = [];
  push(item: T) { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  peek(): T | undefined { return this.items.at(-1); }
}

// Generic fetch helper
async function http<T>(url: string, init?: RequestInit): Promise<T> {
  const res = await fetch(url, init);
  if (!res.ok) throw new Error(res.statusText);
  return res.json() as Promise<T>;
}
const users = await http<User[]>('/api/users');
```

2.8 Built-in Utility Types

```ts
interface Todo { id: number; title: string; done: boolean; tags?: string[] }

type A = Partial<Todo>;                  // all optional
type B = Required<Todo>;                 // all required
type C = Readonly<Todo>;                 // all readonly
type D = Pick<Todo, 'id' | 'title'>;     // subset
type E = Omit<Todo, 'id'>;               // remove keys
type F = Record<'admin' | 'user', number>; // { admin: number; user: number }
type G = Exclude<'a' | 'b' | 'c', 'a'>;  // 'b' | 'c'
type H = Extract<'a' | 'b', 'a' | 'z'>;  // 'a'
type I = NonNullable<string | null>;     // string
type J = ReturnType<typeof setTimeout>;
type K = Parameters<(a: number, b: string) => void>; // [number, string]
type L = Awaited<Promise<Promise<number>>>; // number
type M = InstanceType<typeof Date>;
type N = Uppercase<'hi'>;                 // 'HI' (also Lowercase, Capitalize, Uncapitalize)

// Common patterns
type CreateTodoDto = Omit<Todo, 'id'>;
type UpdateTodoDto = Partial<Omit<Todo, 'id'>> & Pick<Todo, 'id'>;
```

2.9 Advanced Types

`keyof`, `typeof`, indexed access

```ts
const config = { host: 'localhost', port: 5432 } as const;
type Config = typeof config;             // { readonly host: 'localhost'; readonly port: 5432 }
type ConfigKey = keyof Config;           // 'host' | 'port'
type Port = Config['port'];              // 5432

const ROLES = ['admin', 'editor', 'viewer'] as const;
type Role = (typeof ROLES)[number];      // 'admin' | 'editor' | 'viewer'
```

Mapped Types

```ts
type MyReadonly<T> = { readonly [K in keyof T]: T[K] };
type MyPartial<T> = { [K in keyof T]?: T[K] };
type Mutable<T> = { -readonly [K in keyof T]: T[K] };   // remove modifier
type Nullable<T> = { [K in keyof T]: T[K] | null };

// Key remapping with `as`
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};
type UserGetters = Getters<{ name: string; age: number }>;
// { getName: () => string; getAge: () => number }

// Filter keys by value type
type PickByType<T, V> = { [K in keyof T as T[K] extends V ? K : never]: T[K] };
```

Conditional Types and `infer`

```ts
type IsString<T> = T extends string ? true : false;

// Distributive over unions
type ToArray<T> = T extends any ? T[] : never;
type X = ToArray<string | number>;        // string[] | number[]
type NoDistribute<T> = [T] extends [any] ? T[] : never; // (string | number)[]

// infer — extract inner types
type MyReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
type ElementType<T> = T extends (infer U)[] ? U : T;
type UnwrapPromise<T> = T extends Promise<infer U> ? UnwrapPromise<U> : T;

// Deep partial (recursive)
type DeepPartial<T> = {
  [K in keyof T]?: T[K] extends object ? DeepPartial<T[K]> : T[K];
};
```

Template Literal Types

```ts
type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE';
type Route = `/api/${'users' | 'orders'}`;
type Endpoint = `${HttpMethod} ${Route}`;     // 'GET /api/users' | ...

type EventName<T extends string> = `on${Capitalize<T>}`;
type ClickHandler = EventName<'click'>;       // 'onClick'

// Extract route params: '/users/:id/posts/:postId' → 'id' | 'postId'
type Params<S extends string> =
  S extends `${string}:${infer P}/${infer Rest}` ? P | Params<Rest>
  : S extends `${string}:${infer P}` ? P
  : never;
type R = Params<'/users/:id/posts/:postId'>;  // 'id' | 'postId'
```

`satisfies` operator (TS 4.9+)

```ts
type Colors = Record<'primary' | 'secondary', string | [number, number, number]>;

const palette = {
  primary: '#f00',
  secondary: [0, 128, 255],
} satisfies Colors;      // validates shape BUT keeps the narrow inferred type
```

```
palette.primary.toUpperCase();   // OK — known to be string
palette.secondary.map(n => n);   // OK — known to be tuple
```

Branded / Nominal Types

```
type Brand<T, B extends string> = T & { readonly __brand: B };
type UserId = Brand<string, 'UserId'>;
type OrderId = Brand<string, 'OrderId'>;

const toUserId = (s: string) => s as UserId;
function getUser(id: UserId) { /* ... */ }
getUser(toUserId('u_1'));
// getUser('o_1' as OrderId); // Error — prevents mixing IDs
```

2.10 Classes in TypeScript

```
abstract class Repository<T extends { id: number }> {
  protected items = new Map<number, T>();
  constructor(protected readonly tableName: string) {}   // parameter property

  abstract validate(item: T): boolean;

  save(item: T): T {
    if (!this.validate(item)) throw new Error('Invalid');
    this.items.set(item.id, item);
    return item;
  }
  findById(id: number): T | undefined { return this.items.get(id); }
}

interface Product { id: number; name: string; price: number }

class ProductRepo extends Repository<Product> implements Iterable<Product> {
  constructor() { super('products'); }
  validate(p: Product) { return p.price >= 0; }
  override findById(id: number) { return super.findById(id); }
  *[Symbol.iterator]() { yield* this.items.values(); }
}
```

Access modifiers: `public` (default), `protected` (class + subclasses), `private` (TS compile-time only), `#private` (JS runtime-private). `readonly` prevents re-assignment.

2.11 Enums vs Union Literals

```
enum Status { Active = 'ACTIVE', Inactive = 'INACTIVE' }   // emits runtime object
const enum Dir { Up, Down }                                // inlined, no runtime object

// Often preferred: union + as const object (no TS-only runtime emit)
const STATUS = { Active: 'ACTIVE', Inactive: 'INACTIVE' } as const;
type StatusT = (typeof STATUS)[keyof typeof STATUS];       // 'ACTIVE' | 'INACTIVE'
```

Numeric enums are reverse-mapped and accept any number — a common source of bugs. Prefer string enums or union types.

2.12 Decorators (TS 5 Standard Decorators)

```
function log<T extends (...args: any[]) => any>(target: T, ctx: ClassMethodDecoratorContext) {
  return function (this: unknown, ...args: Parameters<T>): ReturnType<T> {
    console.log(`→ ${String(ctx.name)}`, args);
    return target.apply(this, args);
  } as T;
}

class Calculator {
  @log
  add(a: number, b: number) { return a + b; }
}
```

Frameworks like NestJS, TypeORM and Angular use (legacy) `experimentalDecorators` with `emitDecoratorMetadata`.

2.13 Declaration Files and Module Augmentation

```
// types/env.d.ts
declare namespace NodeJS {
  interface ProcessEnv {
    DATABASE_URL: string;
    NODE_ENV: 'development' | 'production' | 'test';
  }
}

// Untyped JS library
declare module 'legacy-lib' {
  export function doThing(x: string): number;
}

// Augment Express Request
declare global {
  namespace Express {
    interface Request { user?: { id: string; role: string } }
  }
}
export {};
```

2.14 TypeScript with React

```
import { useState, useRef, useReducer, createContext, useContext, type ReactNode, type
ComponentProps } from 'react';

type ButtonProps = ComponentProps<'button'> & {
  variant?: 'primary' | 'ghost';
  loading?: boolean;
};

export function Button({ variant = 'primary', loading, children, ...rest }: ButtonProps) {
  return (
    <button className={variant} disabled={loading || rest.disabled} {...rest}>
      {loading ? 'Loading…' : children}
    </button>
  );
}

// State & refs
const [user, setUser] = useState<User | null>(null);
const inputRef = useRef<HTMLInputElement>(null);
```

```tsx
// Events
const onChange = (e: React.ChangeEvent<HTMLInputElement>) => setValue(e.target.value);
const onSubmit = (e: React.FormEvent<HTMLFormElement>) => e.preventDefault();

// Generic component
type ListProps<T> = { items: T[]; render: (item: T) => ReactNode; keyOf: (item: T) => string };
function List<T>({ items, render, keyOf }: ListProps<T>) {
  return <ul>{items.map(i => <li key={keyOf(i)}>{render(i)}</li>)}</ul>;
}

// Typed reducer
type Action = { type: 'inc' } | { type: 'set'; payload: number };
function reducer(state: number, action: Action): number {
  switch (action.type) {
    case 'inc': return state + 1;
    case 'set': return action.payload;
  }
}

// Typed context with safe hook
type AuthCtx = { user: User | null; login: (u: User) => void };
const AuthContext = createContext<AuthCtx || null>(null);
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}
```

2.15 TypeScript with Node/Express

```tsx
import express, { type Request, type Response, type NextFunction } from 'express';
import { z } from 'zod';

const CreateUser = z.object({
  name: z.string().min(2),
  email: z.string().email(),
  age: z.number().int().positive().optional(),
});
type CreateUserDto = z.infer<typeof CreateUser>;   // derive TS type from runtime schema

const app = express();
app.use(express.json());

app.post('/users', (req: Request<{}, {}, CreateUserDto>, res: Response, next: NextFunction) => {
  const parsed = CreateUser.safeParse(req.body);
  if (!parsed.success) return res.status(400).json(parsed.error.flatten());
  res.status(201).json({ id: 1, ...parsed.data });
});
```

Key insight: TS types vanish at runtime, so validate external input (HTTP bodies, env vars, JSON files) with a runtime schema (Zod, Valibot, io-ts) and *infer* the static type from it.

2.16 Type Challenges (Practice)

```tsx
// 1. MyPick
type MyPick<T, K extends keyof T> = { [P in K]: T[P] };

// 2. TupleToUnion
type TupleToUnion<T extends readonly unknown[]> = T[number];

// 3. Last element of tuple
type Last<T extends unknown[]> = T extends [...infer _, infer L] ? L : never;
```

```ts
// 4. DeepReadonly
type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends (...a: any[]) => any ? T[K]
    : T[K] extends object ? DeepReadonly<T[K]> : T[K];
};

// 5. Union to intersection
type UnionToIntersection<U> =
  (U extends any ? (x: U) => void : never) extends (x: infer I) => void ? I : never;

// 6. Required keys of T
type RequiredKeys<T> = { [K in keyof T]-?: {} extends Pick<T, K> ? never : K }[keyof T];
```

2.17 Variance, `any` vs `unknown` vs `never` — Summary

- `any`: disables checking in both directions (assignable to/from everything).
- `unknown`: top type — anything assignable *to* it; must narrow before use.
- `never`: bottom type — assignable to everything, nothing assignable to it (except `never`). Represents impossible states.
- Function parameters are checked **contravariantly** under `strictFunctionTypes` (for function-type properties, not methods).
- `object` = any non-primitive; `{}` = any non-nullish value; `Object` — avoid.

Part 3 — React: Basics to Advanced

3.1 Core Ideas

- **Declarative UI:** describe *what* the UI should look like for a given state; React figures out *how* to update the DOM.
- **Components:** functions that take props and return JSX.
- **Unidirectional data flow:** data goes down via props; events go up via callbacks.
- **UI = f(state):** re-rendering is just calling the component function again.

JSX

JSX compiles to `jsx()` calls (React 17+ automatic runtime), producing plain objects (React elements).

```
const el = <h1 className="title">Hello {name}</h1>;
// compiles to → jsx('h1', { className: 'title', children: ['Hello ', name] })

// Rules: one root (or <>…</> Fragment), className/htmlFor, camelCase events,
// {} for expressions, self-close empty tags, style={{ color: 'red' }}
```

Virtual DOM, Reconciliation and Fiber

1. State changes → React re-renders the component → new element tree.
2. **Reconciliation / diffing:** compares new tree vs previous. Heuristics: different element type ⇒ destroy and rebuild subtree; same type ⇒ update props; lists are matched by `key`.
3. **Commit phase:** applies the minimal DOM mutations, runs layout effects, then passive effects.

Fiber (React 16+) is the reconciliation engine: work is split into units so rendering can be paused, prioritized and resumed — enabling **concurrent rendering** (React 18+).

Render phase (pure, may run multiple times, interruptible) vs **commit phase** (side effects, synchronous).

Keys

```
// Bad — index keys break state when items are reordered/inserted
{todos.map((t, i) => <TodoItem key={i} todo={t} />)}
// Good — stable unique id
{todos.map(t => <TodoItem key={t.id} todo={t} />)}

// Trick: changing `key` resets a component's state entirely
<ProfileForm key={userId} userId={userId} />
```

3.2 Components and Props

```
function Avatar({ src, size = 40, alt = '' }) {
  return <img src={src} width={size} height={size} alt={alt} />;
```

```
}

// children & composition
function Card({ title, children, footer }) {
  return (
    <section className="card">
      <h2>{title}</h2>
      <div>{children}</div>
      {footer && <footer>{footer}</footer>}
    </section>
  );
}

<Card title="Profile" footer={<button>Save</button>}>
  <Avatar src="/me.png" />
</Card>
```

Conditional rendering:

```
{isLoggedIn ? <Dashboard /> : <Login />}
{items.length > 0 && <List items={items} />}   // careful: {0 && <X/>} renders "0"
{items.length ? <List /> : null}
```

Controlled vs uncontrolled inputs:

```
// Controlled — React state is the source of truth
const [email, setEmail] = useState('');
<input value={email} onChange={e => setEmail(e.target.value)} />

// Uncontrolled — DOM holds the value; read via ref / FormData
const ref = useRef(null);
<input defaultValue="x" ref={ref} />
```

3.3 Hooks — Complete Reference

Rules of Hooks: only call at the top level (not in loops/conditions/nested functions) and only from React functions or custom hooks. React relies on call order to associate state with hooks.

useState

```
const [count, setCount] = useState(0);
const [data, setData] = useState(() => expensiveInit()); // lazy initializer — runs once

setCount(count + 1);             // uses the value from this render
setCount(c => c + 1);            // functional update — use when new state depends on old

// State updates are batched (automatic batching in React 18 for all updates)
function handle() {
  setCount(c => c + 1);
  setCount(c => c + 1);          // final: +2 ; with setCount(count + 1) twice → +1
}

// Objects/arrays: always create new references (immutability)
setUser(u => ({ ...u, name: 'New' }));
setTodos(ts => ts.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
setTodos(ts => ts.filter(t => t.id !== id));
```

useEffect

Synchronizes a component with an **external system** (network, subscriptions, timers, DOM APIs). Runs *after* paint.

```
useEffect(() => {
  // runs after every render if no deps array
});

useEffect(() => {
  // runs once after mount
}, []);

useEffect(() => {
  const controller = new AbortController();
  let ignore = false;

  async function load() {
    try {
      const res = await fetch(`/api/users/${userId}`, { signal: controller.signal });
      const json = await res.json();
      if (!ignore) setUser(json);
    } catch (e) {
      if (e.name !== 'AbortError') setError(e);
    }
  }
  load();

  return () => {           // cleanup: runs before next effect & on unmount
    ignore = true;
    controller.abort();
  };
}, [userId]);              // re-run when userId changes (avoids race conditions)
```

In development with `<StrictMode>`, effects run **mount → unmount → mount** to surface missing cleanups.

You might not need an effect:

```
// Bad — derived state via effect causes an extra render
const [fullName, setFullName] = useState('');
useEffect(() => setFullName(`${first} ${last}`), [first, last]);

// Good — compute during render
const fullName = `${first} ${last}`;

// Bad — resetting state on prop change via effect. Good — use `key`.
// Bad — handling user events in effects. Good — do it in the event handler.
```

useLayoutEffect

Same signature as `useEffect`, but fires synchronously **after DOM mutations, before paint**. Use for measuring layout (tooltip positioning) to avoid flicker. Blocks painting — use sparingly.

useRef

```
const inputRef = useRef(null);
useEffect(() => inputRef.current.focus(), []);
<input ref={inputRef} />

// Mutable value that persists across renders WITHOUT causing re-render
```

```
const renderCount = useRef(0);
renderCount.current++;

// Keep the latest value for use in callbacks / intervals
const savedCallback = useRef(callback);
useEffect(() => { savedCallback.current = callback; }, [callback]);
```

useMemo and useCallback

```
// Memoize an expensive computation
const visibleTodos = useMemo(() => filterTodos(todos, tab), [todos, tab]);

// Memoize a function reference (useful when passed to memoized children / effect deps)
const handleSelect = useCallback((id) => setSelected(id), []);

// useCallback(fn, deps) === useMemo(() => fn, deps)
```

Do not memoize everything — it has a cost. Use it when (1) the computation is expensive, (2) referential stability matters for `React.memo` children or effect dependencies. The **React Compiler** (React 19 ecosystem) can auto-memoize.

useReducer

```
const initialState = { items: [], loading: false, error: null };

function reducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':   return { ...state, loading: true, error: null };
    case 'FETCH_SUCCESS': return { ...state, loading: false, items: action.payload };
    case 'FETCH_ERROR':   return { ...state, loading: false, error: action.error };
    case 'REMOVE':        return { ...state, items: state.items.filter(i => i.id !== action.id)
};
    default: throw new Error(`Unknown action ${action.type}`);
  }
}

const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: 'FETCH_START' });
```

Prefer `useReducer` when state has multiple sub-values that change together or next state depends on complex logic.

useContext

```
const ThemeContext = createContext('light');

function App() {
  const [theme, setTheme] = useState('dark');
  const value = useMemo(() => ({ theme, setTheme }), [theme]); // avoid re-render storms
  return (
    <ThemeContext.Provider value={value}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const { theme, setTheme } = useContext(ThemeContext);
  return <button onClick={() => setTheme(t => t === 'dark' ? 'light' : 'dark')}>{theme}
</button>;
}
```

All consumers re-render when the context value changes. Split contexts (state vs dispatch) or use a store library for frequently changing global state.

Other Hooks

```
// useId — stable unique ids (SSR-safe) for accessibility
const id = useId();
<label htmlFor={id}>Email</label><input id={id} />

// useTransition — mark updates as non-urgent (keeps input responsive)
const [isPending, startTransition] = useTransition();
const onChange = (e) => {
  setQuery(e.target.value);                          // urgent
  startTransition(() => setResults(search(e.target.value))); // can be interrupted
};

// useDeferredValue — defer re-rendering of an expensive part
const deferredQuery = useDeferredValue(query);
<SlowList query={deferredQuery} />

// useImperativeHandle — expose a custom API to a parent ref
function FancyInput({ ref }) {       // React 19: ref is a normal prop
  const inner = useRef();
  useImperativeHandle(ref, () => ({ focus: () => inner.current.focus() }));
  return <input ref={inner} />;
}

// useSyncExternalStore — subscribe to external stores safely in concurrent mode
const isOnline = useSyncExternalStore(
  (cb) => { window.addEventListener('online', cb); window.addEventListener('offline', cb);
            return () => { window.removeEventListener('online', cb);
window.removeEventListener('offline', cb); }; },
  () => navigator.onLine,
);
```

React 19 Hooks and APIs

```
// use() — read a promise or context (can be called conditionally)
function Comments({ commentsPromise }) {
  const comments = use(commentsPromise);   // suspends until resolved
  return comments.map(c => <p key={c.id}>{c.text}</p>);
}

// Actions + useActionState — form handling with pending & errors
async function updateName(prevState, formData) {
  const error = await saveName(formData.get('name'));
  return error ? { error } : { error: null };
}
function NameForm() {
  const [state, formAction, isPending] = useActionState(updateName, { error: null });
  return (
    <form action={formAction}>
      <input name="name" />
      <SubmitButton />
      {state.error && <p>{state.error}</p>}
    </form>
  );
}

// useFormStatus — status of the parent <form>
function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>{pending ? 'Saving…' : 'Save'}</button>;
}

// useOptimistic — show the result instantly while the request is in flight
```

```
const [optimisticTodos, addOptimistic] = useOptimistic(todos, (state, newTodo) => [...state,
newTodo]);
```

Other React 19 changes: `ref` as a prop (no more `forwardRef` for new code), `<Context>` usable directly as provider, document metadata (`<title>`, `<meta>`) hoisting, ref cleanup functions.

3.4 Custom Hooks

Custom hooks extract reusable **stateful logic** (not state itself — every call gets its own state).

```
function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: true, error: null });

  useEffect(() => {
    const controller = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error(r.statusText); return r.json(); })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(error => error.name !== 'AbortError' && setState({ data: null, loading: false,
error }));
    return () => controller.abort();
  }, [url]);

  return state;
}

function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try { const v = localStorage.getItem(key); return v ? JSON.parse(v) : initial; }
    catch { return initial; }
  });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(value)); }, [key, value]);
  return [value, setValue];
}

function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current;
}

function useInterval(callback, delay) {
  const saved = useRef(callback);
  useEffect(() => { saved.current = callback; }, [callback]);
  useEffect(() => {
    if (delay == null) return;
    const id = setInterval(() => saved.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}

function useOnClickOutside(ref, handler) {
  useEffect(() => {
    const listener = (e) => { if (!ref.current || ref.current.contains(e.target)) return;
handler(e); };
    document.addEventListener('mousedown', listener);
```

```
    return () => document.removeEventListener('mousedown', listener);
  }, [ref, handler]);
}
```

3.5 Stale Closures — The Most Common Hook Bug

```
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setCount(count + 1);   // BUG: `count` is always 0 (captured from first render)
    }, 1000);
    return () => clearInterval(id);
  }, []);                    // missing dependency

  // FIX 1: functional update → setCount(c => c + 1)
  // FIX 2: add count to deps (interval re-created every tick)
  // FIX 3: keep latest value in a ref
}
```

Always enable `eslint-plugin-react-hooks` (`exhaustive-deps`).

3.6 Performance Optimization

```
// React.memo — skip re-render if props are shallow-equal
const Row = React.memo(function Row({ item, onSelect }) {
  return <li onClick={() => onSelect(item.id)}>{item.name}</li>;
});

function List({ items }) {
  const [selected, setSelected] = useState(null);
  const onSelect = useCallback((id) => setSelected(id), []); // stable ref → memo works
  return <ul>{items.map(i => <Row key={i.id} item={i} onSelect={onSelect} />)}</ul>;
}
```

Why does a component re-render? (1) its state changed, (2) its parent re-rendered, (3) a context it uses changed. Props changing is *not* itself a trigger — parent re-render is.

Techniques:

- Move state down (colocate) so fewer components re-render.
- Lift content up — pass expensive subtrees as `children` (they don't re-render when the wrapper's state changes).
- Code splitting with `React.lazy` + `Suspense`.
- Virtualize long lists (`react-window`, `@tanstack/react-virtual`).
- Debounce input; use `useTransition` / `useDeferredValue`.
- Avoid creating new objects/arrays in props unnecessarily.
- Profile with React DevTools Profiler; check "why did this render".

```
const Settings = lazy(() => import('./Settings'));

<Suspense fallback={<Spinner />}>
  <Settings />
</Suspense>
```

3.7 Advanced Component Patterns

Compound Components

```
const TabsContext = createContext(null);

function Tabs({ defaultIndex = 0, children }) {
  const [active, setActive] = useState(defaultIndex);
  return <TabsContext.Provider value={{ active, setActive }}>{children}</TabsContext.Provider>;
}
Tabs.List = function TabList({ children }) {
  return <div role="tablist">{children}</div>;
};
Tabs.Tab = function Tab({ index, children }) {
  const { active, setActive } = useContext(TabsContext);
  return <button role="tab" aria-selected={active === index} onClick={() => setActive(index)}>
{children}</button>;
};
Tabs.Panel = function Panel({ index, children }) {
  const { active } = useContext(TabsContext);
  return active === index ? <div role="tabpanel">{children}</div> : null;
};

<Tabs>
  <Tabs.List><Tabs.Tab index={0}>A</Tabs.Tab><Tabs.Tab index={1}>B</Tabs.Tab></Tabs.List>
  <Tabs.Panel index={0}>Panel A</Tabs.Panel>
  <Tabs.Panel index={1}>Panel B</Tabs.Panel>
</Tabs>
```

Higher-Order Components (HOC)

```
function withAuth(Component) {
  return function Wrapped(props) {
    const { user } = useAuth();
    if (!user) return <Navigate to="/login" />;
    return <Component {...props} user={user} />;
  };
}
export default withAuth(Dashboard);
```

Render Props

```
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>{render(pos)}</div>;
}
<MouseTracker render={({ x, y }) => <p>{x}, {y}</p>} />
```

Custom hooks have largely replaced HOCs and render props for logic reuse.

Controlled/Uncontrolled hybrid & Portals

```
// Portal — render outside parent DOM (modals, tooltips); events still bubble through React tree
function Modal({ open, onClose, children }) {
  if (!open) return null;
  return createPortal(
    <div className="backdrop" onClick={onClose}>
      <div className="modal" onClick={e => e.stopPropagation()}>{children}</div>
    </div>,
    document.body,
```

```
  );
}
```

3.8 Error Boundaries

Catch rendering errors in the subtree (not in event handlers, async code, or SSR). Must be class components (or use `react-error-boundary`).

```
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };
  static getDerivedStateFromError(error) { return { hasError: true, error }; }
  componentDidCatch(error, info) { logToService(error, info.componentStack); }
  render() {
    if (this.state.hasError) return this.props.fallback ?? <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
```

3.9 Class Component Lifecycle (for legacy code)

Phase	Methods
Mount	`constructor` → `getDerivedStateFromProps` → `render` → `componentDidMount`
Update	`getDerivedStateFromProps` → `shouldComponentUpdate` → `render` → `getSnapshotBeforeUpdate` → `componentDidUpdate`
Unmount	`componentWillUnmount`
Error	`getDerivedStateFromError`, `componentDidCatch`

Hook equivalents: `componentDidMount` ≈ `useEffect(fn, [])`; `componentDidUpdate` ≈ `useEffect(fn, [deps])`; `componentWillUnmount` ≈ cleanup function; `shouldComponentUpdate` ≈ `React.memo`.

3.10 State Management

Kind of state	Tool
Local UI state	`useState` / `useReducer`
Shared across a subtree	Lift state up / Context
Global client state	Redux Toolkit, Zustand, Jotai
Server state (cache, refetch)	TanStack Query, RTK Query, SWR
URL state	Router search params
Form state	React Hook Form + Zod

Redux Toolkit

```
import { configureStore, createSlice, createAsyncThunk } from '@reduxjs/toolkit';

export const fetchTodos = createAsyncThunk('todos/fetch', async () => {
  const res = await fetch('/api/todos');
  return res.json();
```

```
});

const todosSlice = createSlice({
  name: 'todos',
  initialState: { items: [], status: 'idle' },
  reducers: {
    added(state, action) { state.items.push(action.payload); }, // Immer makes "mutation" safe
    toggled(state, action) {
      const t = state.items.find(t => t.id === action.payload);
      if (t) t.done = !t.done;
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchTodos.pending, (s) => { s.status = 'loading'; })
      .addCase(fetchTodos.fulfilled, (s, a) => { s.status = 'idle'; s.items = a.payload; })
      .addCase(fetchTodos.rejected, (s) => { s.status = 'failed'; });
  },
});

export const { added, toggled } = todosSlice.actions;
export const store = configureStore({ reducer: { todos: todosSlice.reducer } });

// In components
const todos = useSelector((s) => s.todos.items);
const dispatch = useDispatch();
dispatch(added({ id: 1, title: 'x', done: false }));
```

Redux principles: single source of truth, state is read-only (change via actions), changes via pure reducers. Middleware (thunk, saga, listener) handle side effects.

Zustand

```
import { create } from 'zustand';
const useCart = create((set, get) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
  total: () => get().items.reduce((sum, i) => sum + i.price, 0),
}));
const items = useCart((s) => s.items);  // selector → re-render only when items change
```

TanStack Query (server state)

```
const { data, isLoading, error } = useQuery({
  queryKey: ['users', page],
  queryFn: () => fetch(`/api/users?page=${page}`).then(r => r.json()),
  staleTime: 60_000,
});

const queryClient = useQueryClient();
const mutation = useMutation({
  mutationFn: (user) => fetch('/api/users', { method: 'POST', body: JSON.stringify(user) }),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['users'] }),
});
```

3.11 Routing (React Router v6/v7)

```
import { createBrowserRouter, RouterProvider, Outlet, Link, NavLink, useParams, useNavigate,
useSearchParams, Navigate } from 'react-router-dom';

const router = createBrowserRouter([
  {
```

```
    path: '/',
    element: <Layout />,
    errorElement: <ErrorPage />,
    children: [
      { index: true, element: <Home /> },
      { path: 'users/:id', element: <UserPage />, loader: ({ params }) => fetchUser(params.id)
},
      { path: 'admin', element: <RequireAuth><Admin /></RequireAuth> },
      { path: '*', element: <NotFound /> },
    ],
  },
]);

function Layout() { return <><nav><NavLink to="/">Home</NavLink></nav><Outlet /></>; }

function UserPage() {
  const { id } = useParams();
  const [params, setParams] = useSearchParams();
  const navigate = useNavigate();
  return <button onClick={() => navigate('/')}>Back</button>;
}

function RequireAuth({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" replace />;
}
```

3.12 Forms — React Hook Form + Zod

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});
type FormValues = z.infer<typeof schema>;

export function LoginForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm<FormValues>({ resolver: zodResolver(schema) });

  const onSubmit = async (values: FormValues) => { await login(values); };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('email')} />
      {errors.email && <span>{errors.email.message}</span>}
      <input type="password" {...register('password')} />
      <button disabled={isSubmitting}>Login</button>
    </form>
  );
}
```

3.13 Rendering Strategies and Server Components

Strategy	When HTML is generated	Use case
CSR	In browser	Dashboards behind auth
SSR	Per request on server	Personalized, SEO pages
SSG	At build time	Docs, blogs, marketing
ISR	Build + revalidate periodically	E-commerce catalogs
Streaming SSR	Server streams chunks with Suspense	Faster TTFB/FCP

Hydration: attaching event listeners to server-rendered HTML. Mismatches (e.g., `Date.now()` or `window` used during render) cause hydration errors.

React Server Components (Next.js App Router):

```
// app/users/page.tsx — Server Component (default): runs only on server, can access DB, zero JS shipped
import { db } from '@/lib/db';
import LikeButton from './LikeButton';

export default async function UsersPage() {
  const users = await db.user.findMany();
  return users.map(u => <div key={u.id}>{u.name} <LikeButton id={u.id} /></div>);
}

// app/users/LikeButton.tsx — Client Component: interactivity, hooks, browser APIs
'use client';
import { useState } from 'react';
export default function LikeButton({ id }: { id: number }) {
  const [liked, setLiked] = useState(false);
  return <button onClick={() => setLiked(!liked)}>{liked ? '♥' : '♡'}</button>;
}

// Server Action
'use server';
export async function createUser(formData: FormData) {
  await db.user.create({ data: { name: String(formData.get('name')) } });
  revalidatePath('/users');
}
```

3.14 Testing React

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('increments counter', async () => {
  render(<Counter />);
  await userEvent.click(screen.getByRole('button', { name: /increment/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});

test('loads users', async () => {
  global.fetch = vi.fn().mockResolvedValue({ ok: true, json: async () => [{ id: 1, name: 'A' }]
});
  render(<Users />);
  expect(await screen.findByText('A')).toBeInTheDocument(); // findBy waits
});

// Testing a custom hook
```

```
import { renderHook, act } from '@testing-library/react';
const { result } = renderHook(() => useCounter());
act(() => result.current.inc());
expect(result.current.count).toBe(1);
```

Query priority: `getByRole` > `getByLabelText` > `getByPlaceholderText` > `getByText` > `getByTestId`. Mock network with MSW for realistic tests.

3.15 Interview Questions Checklist

- Difference between state and props; why state is immutable.
- What triggers a re-render; how `React.memo`, `useMemo`, `useCallback` relate.
- Why keys matter; why index keys are bad.
- `useEffect` vs `useLayoutEffect`; effect cleanup; StrictMode double invocation.
- Controlled vs uncontrolled components.
- Context performance pitfalls.
- How batching works in React 18.
- What is Fiber / concurrent rendering / Suspense.
- SSR vs CSR vs SSG; hydration; Server Components vs Client Components.
- How to prevent race conditions in data fetching.
- Lifting state up vs global stores; server state vs client state.

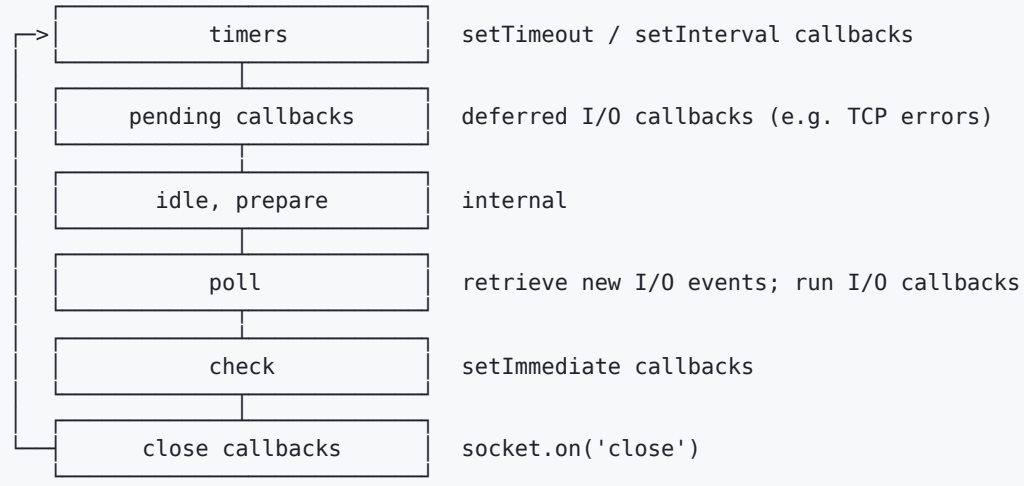
Part 4 — Node.js: Basics to Advanced

4.1 Architecture

Node.js = **V8** (JS engine) + **libuv** (C library: event loop, thread pool, async I/O) + core bindings (fs, net, crypto, zlib, http).

- **Single-threaded event loop** for JavaScript execution.
- **Non-blocking I/O:** network I/O uses OS async mechanisms (epoll/kqueue/IOCP); file system, DNS lookup, crypto and zlib use the **libuv thread pool** (default 4 threads, `UV_THREADPOOL_SIZE`).
- Great for I/O-bound workloads (APIs, real-time); CPU-heavy work blocks the loop → use worker threads or offload.

4.2 The Node.js Event Loop Phases



Between every callback: process.nextTick queue, then Promise microtask queue

```
const fs = require('fs');

setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));
console.log('sync');
// sync, nextTick, promise, then timeout/immediate (order not guaranteed in main module)

fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout in I/O'), 0);
  setImmediate(() => console.log('immediate in I/O')); // ALWAYS first inside an I/O callback
});
```

`process.nextTick` runs before promise microtasks. Recursive `nextTick` can starve the event loop.

4.3 Modules

```
// CommonJS
const path = require('node:path');
module.exports = { sum };          // or exports.sum = sum (never reassign `exports`)
// Module wrapper: (function (exports, require, module, __filename, __dirname) { ... })
// Modules are cached after first require → effectively singletons

// ESM — "type": "module" in package.json or .mjs
import { readFile } from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
const __dirname = path.dirname(fileURLToPath(import.meta.url)); // or import.meta.dirname (Node 20.11+)
```

package.json essentials: `"type"`, `"main"`, `"exports"` (entry points & conditional exports), `"scripts"`, `"engines"`, `dependencies` vs `devDependencies` vs `peerDependencies`. Semver: `^1.2.3` (minor+patch), `~1.2.3` (patch only). Commit lock files; use `npm ci` in CI.

4.4 Core Modules

File System

```
import { readFile, writeFile, mkdir, readdir, stat, rm } from 'node:fs/promises';
import { existsSync, createReadStream } from 'node:fs';

await mkdir('./out', { recursive: true });
await writeFile('./out/a.json', JSON.stringify({ a: 1 }, null, 2));
const data = JSON.parse(await readFile('./out/a.json', 'utf8'));
for (const entry of await readdir('.', { withFileTypes: true })) {
  console.log(entry.name, entry.isDirectory());
}
// Avoid *Sync methods in request handlers — they block the event loop
```

path, os, url, crypto

```
import path from 'node:path';
path.join('a', 'b', '../c');          // 'a/c'
path.resolve('src', 'index.js');      // absolute path
path.extname('file.tar.gz');          // '.gz'

import os from 'node:os';
os.cpus().length; os.freemem(); os.platform();

import crypto from 'node:crypto';
crypto.randomUUID();
crypto.createHash('sha256').update('data').digest('hex');
crypto.randomBytes(32).toString('hex');
// Password hashing: scrypt (built-in) or bcrypt/argon2
const salt = crypto.randomBytes(16);
const hash = crypto.scryptSync('password', salt, 64);
crypto.timingSafeEqual(hashA, hashB); // constant-time comparison
```

4.5 EventEmitter

```
import { EventEmitter, once } from 'node:events';
```

```
class OrderService extends EventEmitter {
  place(order) {
    // ... save
    this.emit('order:placed', order);
  }
}

const svc = new OrderService();
svc.on('order:placed', (o) => console.log('send email', o.id));
svc.once('order:placed', () => console.log('first order only'));
svc.on('error', (err) => console.error(err)); // unhandled 'error' event crashes the process!
svc.place({ id: 1 });

const [value] = await once(svc, 'order:placed'); // promise-based wait
```

Listeners run **synchronously** in registration order. Default max listeners warning at 10 (`setMaxListeners`).

4.6 Buffers and Streams

Buffer: fixed-size raw binary data outside the V8 heap.

```
const buf = Buffer.from('hello', 'utf8');
buf.toString('base64');     // 'aGVsbG8='
Buffer.alloc(10);           // zero-filled
Buffer.concat([b1, b2]);
```

Streams process data in chunks → constant memory for huge files. Four types: **Readable**, **Writable**, **Duplex** (socket), **Transform** (zlib, crypto).

```
import { createReadStream, createWriteStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';
import { Transform } from 'node:stream';
import zlib from 'node:zlib';

// Compress a 10GB file using ~constant memory; pipeline handles errors + cleanup
await pipeline(
  createReadStream('big.log'),
  zlib.createGzip(),
  createWriteStream('big.log.gz'),
);

// Custom transform: uppercase
const upper = new Transform({
  transform(chunk, enc, cb) { cb(null, chunk.toString().toUpperCase()); },
});

// Async iteration over a readable
import readline from 'node:readline';
const rl = readline.createInterface({ input: createReadStream('data.csv') });
for await (const line of rl) { /* process each line */ }

// Streaming an HTTP response
app.get('/download', (req, res) => {
  res.setHeader('Content-Type', 'text/csv');
  createReadStream('report.csv').pipe(res);
});
```

Backpressure: when a writable can't keep up, `write()` returns `false`; wait for `'drain'`. `pipe()` / `pipeline()` handle this automatically.

4.7 HTTP Server from Scratch

```
import http from 'node:http';

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);

  if (req.method === 'GET' && url.pathname === '/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ ok: true }));
  }

  if (req.method === 'POST' && url.pathname === '/echo') {
    let body = '';
    for await (const chunk of req) body += chunk;
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(body);
  }

  res.writeHead(404).end('Not found');
});

server.listen(3000, () => console.log('listening on 3000'));
```

4.8 Express.js In Depth

```
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';

const app = express();

// Global middleware (order matters!)
app.use(helmet());                                    // security headers
app.use(cors({ origin: ['https://app.example.com'], credentials: true }));
app.use(express.json({ limit: '1mb' }));
app.use(express.urlencoded({ extended: true }));
app.use(rateLimit({ windowMs: 15 * 60_000, limit: 100 }));

// Custom middleware: request logging
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => console.log(`${req.method} ${req.originalUrl} ${res.statusCode}
${Date.now() - start}ms`));
  next();
});

// Router
const users = express.Router();
users.get('/', async (req, res) => {
  const { page = 1, limit = 20 } = req.query;
  res.json(await UserService.list({ page: +page, limit: +limit }));
});
users.get('/:id', async (req, res, next) => {
  try {
    const user = await UserService.get(req.params.id);
    if (!user) throw new NotFoundError('User not found');
    res.json(user);
  } catch (e) { next(e); }   // Express 5 forwards rejected promises automatically
});
app.use('/api/users', users);

// 404 handler
app.use((req, res) => res.status(404).json({ message: 'Route not found' }));
```

```
// Error-handling middleware: 4 arguments, registered LAST
class AppError extends Error {
  constructor(message, status = 500) { super(message); this.status = status; }
}
class NotFoundError extends AppError { constructor(m) { super(m, 404); } }

app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) console.error(err);
  res.status(status).json({
    message: status >= 500 ? 'Internal Server Error' : err.message,
    ...(process.env.NODE_ENV !== 'production' && { stack: err.stack }),
  });
});

app.listen(process.env.PORT ?? 3000);
```

Async wrapper (Express 4)

```
const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res,
next)).catch(next);
router.get('/', asyncHandler(async (req, res) => res.json(await svc.list())));
```

Layered Architecture

```
src/
  routes/        → URL → controller mapping
  controllers/   → parse req, call service, shape res (no business logic)
  services/      → business logic, transactions
  repositories/  → DB access (SQL/ORM)
  models/        → schemas / entities
  middlewares/   → auth, validation, error handling
  config/        → env parsing & validation
  utils/
```

4.9 Authentication and Authorization

JWT

```
import jwt from 'jsonwebtoken';
import bcrypt from 'bcrypt';

async function login(req, res) {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ message: 'Invalid credentials' });
  }
  const accessToken = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, {
expiresIn: '15m' });
  const refreshToken = jwt.sign({ sub: user.id }, process.env.REFRESH_SECRET, { expiresIn: '7d'
});

  res.cookie('refreshToken', refreshToken, {
    httpOnly: true, secure: true, sameSite: 'strict', maxAge: 7 * 24 * 3600 * 1000,
  });
  res.json({ accessToken });
}

function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
```

```
  if (!token) return res.status(401).json({ message: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
}

const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next() : res.status(403).json({ message: 'Forbidden' });

router.delete('/:id', authenticate, authorize('admin'), deleteUser);
```

JWT structure: `header.payload.signature` (base64url). Payload is *encoded, not encrypted* — never store secrets in it. Short-lived access token + rotating refresh token; store refresh token in `HttpOnly` cookie. **Sessions** are stateful (server store like Redis), easy to revoke; JWTs are stateless, harder to revoke (use denylist or short expiry).

401 vs 403: 401 = not authenticated; 403 = authenticated but not allowed.

4.10 Databases from Node

```
// PostgreSQL with pg — ALWAYS parameterize (prevents SQL injection)
import pg from 'pg';
const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL, max: 10 });

const { rows } = await pool.query('SELECT * FROM users WHERE email = $1', [email]);

// Transaction
const client = await pool.connect();
try {
  await client.query('BEGIN');
  await client.query('UPDATE accounts SET balance = balance - $1 WHERE id = $2', [amt, from]);
  await client.query('UPDATE accounts SET balance = balance + $1 WHERE id = $2', [amt, to]);
  await client.query('COMMIT');
} catch (e) {
  await client.query('ROLLBACK');
  throw e;
} finally {
  client.release();
}

// Mongoose (MongoDB)
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true, lowercase: true },
  name: String,
  posts: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Post' }],
}, { timestamps: true });
userSchema.index({ email: 1 });
userSchema.pre('save', async function () {
  if (this.isModified('password')) this.password = await bcrypt.hash(this.password, 12);
});
const User = mongoose.model('User', userSchema);
await User.find({ name: /an/i }).populate('posts').lean().limit(10);

// Prisma
const user = await prisma.user.create({ data: { email, posts: { create: [{ title: 'Hi' }] } }
});
const users = await prisma.user.findMany({ where: { email: { contains: '@x.com' } }, include: {
posts: true } });
await prisma.$transaction([prisma.a.update(...), prisma.b.update(...)]);
```

4.11 Caching with Redis

```
import { createClient } from 'redis';
const redis = createClient({ url: process.env.REDIS_URL });
await redis.connect();

// Cache-aside pattern
async function getProduct(id) {
  const key = `product:${id}`;
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  const product = await db.product.findUnique({ where: { id } });
  await redis.set(key, JSON.stringify(product), { EX: 300 }); // TTL 5 min
  return product;
}
// Invalidate on write: await redis.del(`product:${id}`)
```

Use cases: caching, sessions, rate limiting (`INCR` + `EXPIRE`), pub/sub, queues (BullMQ), distributed locks.

4.12 Scaling Node.js

Cluster (multi-process on one machine)

```
import cluster from 'node:cluster';
import os from 'node:os';

if (cluster.isPrimary) {
  for (let i = 0; i < os.availableParallelism(); i++) cluster.fork();
  cluster.on('exit', (w) => { console.log(`worker ${w.process.pid} died`); cluster.fork(); });
} else {
  startServer();  // workers share the same port
}
// In production often PM2 (`pm2 start app.js -i max`) or Kubernetes replicas instead.
```

Worker Threads (CPU-bound work)

```
// main.js
import { Worker } from 'node:worker_threads';
function runHeavy(n) {
  return new Promise((resolve, reject) => {
    const w = new Worker(new URL('./fib-worker.js', import.meta.url), { workerData: n });
    w.once('message', resolve);
    w.once('error', reject);
  });
}

// fib-worker.js
import { parentPort, workerData } from 'node:worker_threads';
const fib = (n) => (n < 2 ? n : fib(n - 1) + fib(n - 2));
parentPort.postMessage(fib(workerData));
```

	`cluster`	`worker_threads`	`child_process`
Unit	Process	Thread	Process
Memory	Separate	Shared possible (`SharedArrayBuffer`)	Separate
Use	Scale HTTP across cores	CPU-intensive JS	Run external commands/scripts

child_process

```
import { exec, spawn, execFile } from 'node:child_process';
spawn('ls', ['-la'], { stdio: 'inherit' });   // streams, for long-running/large output
execFile('git', ['status'], (err, stdout) => console.log(stdout)); // safer than exec (no shell)
```

4.13 Background Jobs and Queues

```
import { Queue, Worker } from 'bullmq';
const emailQueue = new Queue('emails', { connection });

await emailQueue.add('welcome', { userId: 1 }, { attempts: 3, backoff: { type: 'exponential',
delay: 1000 } });

new Worker('emails', async (job) => {
  await sendWelcomeEmail(job.data.userId);
}, { connection, concurrency: 5 });
```

4.14 Real-time: WebSockets / Socket.IO

```
import { Server } from 'socket.io';
const io = new Server(httpServer, { cors: { origin: '*' } });

io.use((socket, next) => {               // auth middleware
  try { socket.user = jwt.verify(socket.handshake.auth.token, SECRET); next(); }
  catch { next(new Error('unauthorized')); }
});

io.on('connection', (socket) => {
  socket.join(`room:${socket.user.sub}`);
  socket.on('chat:message', (msg) => io.to(msg.roomId).emit('chat:message', msg));
  socket.on('disconnect', () => console.log('bye'));
});
// Scale horizontally with the Redis adapter.
```

Alternatives: Server-Sent Events (one-way server → client over HTTP), long polling.

4.15 Error Handling and Process Management

```
// Operational errors (expected: invalid input, timeouts) → handle & respond
// Programmer errors (bugs) → crash and let the process manager restart

process.on('unhandledRejection', (reason) => { logger.error(reason); throw reason; });
process.on('uncaughtException', (err) => { logger.fatal(err); process.exit(1); });

// Graceful shutdown
const server = app.listen(3000);
```

```
async function shutdown(signal) {
  console.log(`${signal} received`);
  server.close(async () => {          // stop accepting new connections
    await pool.end();
    await redis.quit();
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 10_000).unref(); // force exit
}
process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);
```

4.16 Configuration and Logging

```
// Validate env at startup — fail fast
import { z } from 'zod';
const Env = z.object({
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development'),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
});
export const env = Env.parse(process.env);
// Node 20.6+: node --env-file=.env app.js

// Structured logging with pino
import pino from 'pino';
const logger = pino({ level: env.NODE_ENV === 'production' ? 'info' : 'debug' });
logger.info({ userId: 1, orderId: 9 }, 'order placed');
```

4.17 Security Checklist

- Validate and sanitize all input (Zod/Joi); parameterized queries (SQL injection); avoid `eval`.
- `helmet` for headers; strict CORS; rate limiting; request size limits.
- Hash passwords with bcrypt/argon2; never log secrets/tokens.
- NoSQL injection: reject objects where strings are expected (`{ "$gt": "" }`).
- Prototype pollution: don't deep-merge untrusted objects.
- `npm audit`, Dependabot; pin versions; lockfiles.
- HTTPS, secure cookies (`HttpOnly`, `Secure`, `SameSite`), CSRF protection for cookie auth.
- Avoid ReDoS (catastrophic regex backtracking); avoid blocking sync calls.

4.18 Performance

- Never block the event loop: avoid sync fs/crypto, giant `JSON.parse`, heavy loops in handlers.
- Use streams for large data; pagination for large queries.
- Connection pooling for DBs; HTTP keep-alive agents.
- Cache (in-memory LRU, Redis, HTTP cache headers / ETags), compression.
- Profile: `node --inspect`, `--prof`, `clinic.js`, `0x` flame graphs; measure event loop lag (`perf_hooks.monitorEventLoopDelay`).
- Memory leaks: heap snapshots in Chrome DevTools; watch for growing global caches, listeners, closures.

4.19 Testing Node

```
// Jest/Vitest + Supertest
import request from 'supertest';
import { app } from '../src/app';

describe('GET /api/users/:id', () => {
  it('returns 404 for unknown user', async () => {
    const res = await request(app).get('/api/users/999');
    expect(res.status).toBe(404);
  });

  it('returns user', async () => {
    vi.spyOn(UserService, 'get').mockResolvedValue({ id: 1, name: 'A' });
    const res = await request(app).get('/api/users/1').set('Authorization', `Bearer ${token}`);
    expect(res.body).toMatchObject({ id: 1 });
  });
});

// Built-in test runner (Node 18+)
import { test } from 'node:test';
import assert from 'node:assert/strict';
test('adds', () => assert.equal(1 + 1, 2));
```

4.20 REST API Design Quick Reference

Method	Purpose	Idempotent	Typical success
GET	Read	Yes	200
POST	Create	No	201 + Location
PUT	Replace	Yes	200 / 204
PATCH	Partial update	Not necessarily	200
DELETE	Remove	Yes	204

Status codes: 200 OK, 201 Created, 204 No Content, 301/302 Redirect, 304 Not Modified, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Error, 502 Bad Gateway, 503 Service Unavailable.

Good practices: plural nouns (`/users/42/orders`), versioning (`/v1`), pagination (`?page=&limit=` or cursor-based), filtering/sorting via query params, consistent error shape, idempotency keys for payments, OpenAPI documentation.

Part 5 — Python: Basics to Advanced

5.1 How Python Runs

CPython compiles source to **bytecode** (`.pyc` in `__pycache__`) and executes it on a stack-based virtual machine. Everything is an **object** with an identity (`id()`), a type and a value. Python is **dynamically** and **strongly** typed (`'1' + 1` raises `TypeError`).

Names are references. Assignment binds a name to an object; it never copies.

```
a = [1, 2]
b = a           # same object
b.append(3)
print(a)        # [1, 2, 3]
a is b          # True  (identity)
a == [1, 2, 3]  # True  (equality)

x = 256; y = 256; x is y   # True — small ints (-5..256) are cached; never rely on `is` for values
```

Mutable: `list`, `dict`, `set`, `bytearray`, user objects. **Immutable:** `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `bytes`, `None`.

5.2 Core Data Types

Strings

```
s = "Hello, World"
s.lower(), s.upper(), s.title()
s.split(", ")              # ['Hello', 'World']
", ".join(["a", "b"])      # 'a, b'
s.strip(), s.replace("l", "L"), s.startswith("He"), s.find("o")
s[::-1]                    # reverse
s[0:5]                     # slicing [start:stop:step]

name, score = "Anish", 92.456
f"{name:>10} | {score:.2f} | {score:08.3f} | {1_000_000:,}"
f"{name=}"                 # "name='Anish'"  (debug)
```

Lists, Tuples, Sets, Dicts

```
nums = [3, 1, 2]
nums.append(4); nums.extend([5, 6]); nums.insert(0, 0)
nums.pop(); nums.remove(3)
sorted(nums, reverse=True)       # new list
nums.sort(key=lambda x: -x)      # in-place

t = (1, 2, 3)
a, *rest = t                     # unpacking: a=1, rest=[2,3]
single = (1,)                    # comma makes the tuple

s = {1, 2, 3}
s | {4}, s & {1, 9}, s - {1}, s ^ {1, 9}   # union, intersection, difference, sym. diff

d = {"a": 1, "b": 2}
```

```
d.get("z", 0)                    # default if missing
d.setdefault("c", []).append(1)
d.items(), d.keys(), d.values()
merged = d | {"b": 99}           # 3.9+ merge
{**d, "x": 1}                    # unpack merge
del d["a"]; d.pop("b", None)
```

Time complexity

Operation	list	dict / set
Index / lookup	O(1) index, O(n) `in`	O(1) average
Append	O(1) amortized	O(1)
Insert/delete at start	O(n)	—
Sort	O(n log n)	—

Use `collections.deque` for O(1) appends/pops at both ends.

5.3 Control Flow and Comprehensions

```
# Comprehensions
squares = [x * x for x in range(10) if x % 2 == 0]
matrix_t = [[row[i] for row in matrix] for i in range(len(matrix[0]))]
word_len = {w: len(w) for w in ["hi", "there"]}
unique = {x % 3 for x in range(10)}
gen = (x * x for x in range(10**9))   # generator expression — lazy, O(1) memory

# for/else — else runs if loop wasn't broken
for n in nums:
    if n < 0:
        break
else:
    print("no negatives")

# Walrus operator (3.8+)
if (n := len(data)) > 10:
    print(f"too long: {n}")

# enumerate / zip
for i, (name, age) in enumerate(zip(names, ages), start=1):
    print(i, name, age)

# Structural pattern matching (3.10+)
def handle(event):
    match event:
        case {"type": "click", "x": x, "y": y}:
            return f"click at {x},{y}"
        case {"type": "key", "key": "Enter" | "Return"}:
            return "submit"
        case [first, *others]:
            return f"list starting with {first}"
        case Point(x=0, y=0):
            return "origin"
        case _:
            return "unknown"
```

5.4 Functions

```python
def greet(name, greeting="Hello", *args, sep=", ", **kwargs):
    return f"{greeting}{sep}{name}"

greet("A", "Hi", sep=" ")

# Positional-only (/) and keyword-only (*) parameters
def f(a, b, /, c, *, d):
    ...
f(1, 2, 3, d=4)       # a,b positional-only; d keyword-only

# Mutable default argument pitfall
def add_item(item, bucket=[]):     # BAD — same list shared across calls
    bucket.append(item)
    return bucket

def add_item(item, bucket=None):   # GOOD
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

# Lambdas, map/filter, functools
from functools import reduce, partial
square = lambda x: x * x
list(map(square, [1, 2]))
list(filter(lambda x: x > 1, [1, 2, 3]))
reduce(lambda acc, x: acc + x, [1, 2, 3], 0)
int_from_bin = partial(int, base=2)
int_from_bin("101")   # 5
```

Scope — LEGB

Lookup order: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

```python
count = 0
def outer():
    total = 0
    def inner():
        nonlocal total       # modify enclosing variable
        global count         # modify global
        total += 1
        count += 1
        return total
    return inner

counter = outer()
counter(); counter()         # 2  — closure keeps `total` alive
```

5.5 Decorators

A decorator is a callable that takes a function and returns a new function.

```python
import time
import functools

def timer(func):
    @functools.wraps(func)               # preserves __name__, __doc__
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
```

```
        print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

# Decorator with arguments → three levels
def retry(times=3, exceptions=(Exception,), delay=0.5):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions:
                    if attempt == times:
                        raise
                    time.sleep(delay * 2 ** (attempt - 1))
        return wrapper
    return decorator

@timer
@retry(times=5, exceptions=(ConnectionError,))
def call_api():
    ...

# Built-in caching
@functools.lru_cache(maxsize=128)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

# Class-based decorator
class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func, self.calls = func, 0
    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.func(*args, **kwargs)
```

Decorators are applied bottom-up: `@a @b def f` ⇒ `f = a(b(f))`.

5.6 Iterators and Generators

```
# Iterator protocol: __iter__ returns self, __next__ raises StopIteration
class Countdown:
    def __init__(self, start): self.current = start
    def __iter__(self): return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

# Generator — much simpler
def countdown(n):
    while n > 0:
        yield n
        n -= 1

def read_large_file(path):
    with open(path) as f:
        for line in f:          # file objects are lazy iterators
            yield line.rstrip("\n")

# Pipeline of generators — processes GBs with constant memory
lines = read_large_file("app.log")
errors = (l for l in lines if "ERROR" in l)
first_10 = list(itertools.islice(errors, 10))
```

```python
# yield from — delegate to sub-generator
def flatten(items):
    for x in items:
        if isinstance(x, list):
            yield from flatten(x)
        else:
            yield x

# send() — coroutine-style generator
def averager():
    total = count = 0
    avg = None
    while True:
        value = yield avg
        total += value; count += 1
        avg = total / count
g = averager(); next(g); g.send(10); g.send(20)  # 15.0
```

itertools / collections

```python
import itertools as it
from collections import Counter, defaultdict, deque, namedtuple, OrderedDict, ChainMap

it.chain([1], [2]); it.product("ab", repeat=2); it.permutations([1, 2, 3], 2)
it.combinations([1, 2, 3], 2); it.groupby(sorted(data, key=k), key=k)
it.accumulate([1, 2, 3]); it.count(10); it.cycle("ab"); it.batched(range(10), 3)  # 3.12

Counter("mississippi").most_common(2)     # [('i', 4), ('s', 4)]
graph = defaultdict(list); graph["a"].append("b")
q = deque(maxlen=3); q.appendleft(1); q.rotate(1)
Point = namedtuple("Point", "x y")
```

5.7 Object-Oriented Python

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    count = 0                           # class attribute (shared)

    def __init__(self, name: str):
        self.name = name                # instance attribute
        Shape.count += 1

    @abstractmethod
    def area(self) -> float: ...

    @classmethod
    def total(cls) -> int:              # receives the class — alternative constructors
        return cls.count

    @staticmethod
    def unit() -> str:                  # no self/cls — utility namespaced in class
        return "cm"

    def __str__(self):  return f"{self.name}: {self.area():.2f}{self.unit()}²"   # user-facing
    def __repr__(self): return f"{type(self).__name__}(name={self.name!r})"      # dev-facing


class Rectangle(Shape):
    def __init__(self, w: float, h: float):
        super().__init__("rect")
        self._w = w                     # "protected" by convention
        self.__secret = 1               # name-mangled → _Rectangle__secret
        self.h = h
```

```
    @property
    def w(self):                        # getter
        return self._w

    @w.setter
    def w(self, value):
        if value <= 0:
            raise ValueError("width must be positive")
        self._w = value

    def area(self):
        return self._w * self.h

    @classmethod
    def square(cls, side):
        return cls(side, side)


r = Rectangle.square(3)
r.w = 4
print(r, repr(r), Shape.total())
```

Dunder (Magic) Methods

```
class Money:
    __slots__ = ("amount", "currency")         # no __dict__ → less memory, faster attrs

    def __init__(self, amount, currency="INR"):
        self.amount, self.currency = amount, currency
    def __add__(self, other):                  # +
        self._check(other)
        return Money(self.amount + other.amount, self.currency)
    def __eq__(self, other):                   # ==
        return (self.amount, self.currency) == (other.amount, other.currency)
    def __lt__(self, other): return self.amount < other.amount
    def __hash__(self): return hash((self.amount, self.currency))  # needed if __eq__ defined
    def __bool__(self): return self.amount != 0
    def __repr__(self): return f"Money({self.amount}, {self.currency!r})"
    def _check(self, o):
        if o.currency != self.currency:
            raise ValueError("currency mismatch")

# Container protocol
class Playlist:
    def __init__(self, songs): self._songs = list(songs)
    def __len__(self): return len(self._songs)
    def __getitem__(self, i): return self._songs[i]        # enables indexing, slicing,
iteration
    def __contains__(self, s): return s in self._songs
    def __iter__(self): return iter(self._songs)
    def __call__(self): return self._songs[0]              # instance is callable
```

Inheritance and MRO

```
class A:
    def hello(self): return "A"
class B(A):
    def hello(self): return "B" + super().hello()
class C(A):
    def hello(self): return "C" + super().hello()
class D(B, C):
    pass

D().hello()   # 'BCA'
D.__mro__     # (D, B, C, A, object) — C3 linearization; super() follows the MRO, not the parent
```

Mixins add reusable behaviour (heavily used in Django CBVs: `LoginRequiredMixin`, etc.).

Dataclasses

```python
from dataclasses import dataclass, field, asdict

@dataclass(frozen=True, slots=True, order=True)
class Employee:
    name: str
    salary: float = 0.0
    skills: list[str] = field(default_factory=list)   # never use a mutable default directly

    def __post_init__(self):
        if self.salary < 0:
            raise ValueError("salary < 0")

e = Employee("A", 100, ["python"])
asdict(e)
```

Also know: `typing.NamedTuple`, `enum.Enum`, `Pydantic` models (validation; used by FastAPI, Django Ninja).

5.8 Context Managers

```python
with open("a.txt") as f, open("b.txt", "w") as g:
    g.write(f.read())

# Class-based
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        return False       # True would suppress the exception

# Generator-based
from contextlib import contextmanager, suppress

@contextmanager
def db_transaction(conn):
    cur = conn.cursor()
    try:
        yield cur
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()

with suppress(FileNotFoundError):
    os.remove("tmp.txt")
```

5.9 Exceptions

```python
class AppError(Exception):
    """Base for app errors."""

class NotFound(AppError):
    def __init__(self, resource, id_):
```

```
        super().__init__(f"{resource} {id_} not found")
        self.resource, self.id = resource, id_

try:
    value = int(user_input)
except (ValueError, TypeError) as e:
    logger.warning("bad input: %s", e)
    raise AppError("invalid number") from e      # exception chaining
else:
    print("runs only if no exception")
finally:
    print("always runs")

# Exception groups (3.11+)
try:
    raise ExceptionGroup("many", [ValueError(1), TypeError(2)])
except* ValueError as eg:
    print("value errors", eg.exceptions)
```

EAFP ("easier to ask forgiveness than permission") is idiomatic: try the operation and catch the exception rather than checking first.

5.10 Type Hints

```
from typing import TypedDict, Protocol, Literal, Callable, TypeVar, Generic, Optional, Any,
Final, overload
from collections.abc import Iterable, Iterator, Sequence, Mapping

def total(prices: list[float], discount: float | None = None) -> float: ...

Status = Literal["active", "inactive"]

class UserDict(TypedDict, total=False):
    id: int
    name: str

class SupportsClose(Protocol):          # structural typing (duck typing for type checkers)
    def close(self) -> None: ...

T = TypeVar("T")
def first(items: Sequence[T]) -> T | None:
    return items[0] if items else None

# 3.12 generic syntax
def last[T](items: list[T]) -> T: return items[-1]
class Box[T]:
    def __init__(self, item: T): self.item = item
type Pair[T] = tuple[T, T]

Handler = Callable[[str, int], bool]
MAX_RETRIES: Final = 3
```

Check with `mypy` or `pyright`. Type hints are not enforced at runtime (use Pydantic for that).

5.11 Concurrency: Threads, Processes, Asyncio

GIL (Global Interpreter Lock): in standard CPython only one thread executes Python bytecode at a time. Consequences:

- **I/O-bound** → threads or `asyncio` work well (GIL released during I/O).

- **CPU-bound** → use `multiprocessing` / `ProcessPoolExecutor` (separate interpreters), or C extensions (NumPy). Python 3.13+ offers an experimental **free-threaded** (no-GIL) build.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor, as_completed
import requests

urls = [...]
with ThreadPoolExecutor(max_workers=10) as pool:              # I/O-bound
    futures = {pool.submit(requests.get, u, timeout=5): u for u in urls}
    for fut in as_completed(futures):
        print(futures[fut], fut.result().status_code)

def crunch(n): return sum(i * i for i in range(n))
if __name__ == "__main__":
    with ProcessPoolExecutor() as pool:                        # CPU-bound
        print(list(pool.map(crunch, [10**6] * 8)))

# Thread safety
import threading
lock = threading.Lock()
counter = 0
def incr():
    global counter
    with lock:
        counter += 1
```

asyncio

```
import asyncio
import aiohttp

async def fetch(session, url):
    async with session.get(url) as resp:
        return await resp.json()

async def main():
    async with aiohttp.ClientSession() as session:
        # run concurrently
        results = await asyncio.gather(*(fetch(session, u) for u in urls),
return_exceptions=True)

        # limit concurrency
        sem = asyncio.Semaphore(5)
        async def bounded(u):
            async with sem:
                return await fetch(session, u)

        # structured concurrency (3.11+)
        async with asyncio.TaskGroup() as tg:
            t1 = tg.create_task(bounded(urls[0]))
            t2 = tg.create_task(bounded(urls[1]))

        # timeout
        async with asyncio.timeout(3):
            await fetch(session, urls[2])

        # run blocking code without blocking the loop
        data = await asyncio.to_thread(blocking_io_function)

asyncio.run(main())
```

Never call blocking functions (`time.sleep` , `requests.get`) inside `async def` — use `await asyncio.sleep` , `httpx.AsyncClient` , etc.

5.12 Modules, Packages and Environments

```
# package/
#   __init__.py
#   utils.py
from package.utils import helper       # absolute import
from .utils import helper              # relative import (inside package)

if __name__ == "__main__":             # runs only when executed directly
    main()
```

- Virtual envs: `python -m venv .venv && source .venv/bin/activate`; tools: `pip`, `pip-tools`, `poetry`, `uv`.
- `pyproject.toml` is the modern project config (dependencies, build, ruff/black/mypy config).
- Formatting & linting: `ruff`, `black`, `isort`; testing: `pytest`.

5.13 Memory Model and Internals

- **Reference counting** frees objects immediately when the count hits zero; a **cyclic garbage collector** (`gc` module) handles reference cycles.
- `sys.getrefcount(obj)`, `sys.getsizeof(obj)`, `tracemalloc` for leak hunting.
- **Shallow vs deep copy:**

```
import copy
a = [[1, 2], [3]]
b = copy.copy(a)        # new outer list, same inner lists
c = copy.deepcopy(a)    # fully independent
b[0].append(99)         # affects a, not c
```

- **Interning:** short strings/identifiers may be interned; `sys.intern()`.
- **Descriptors:** objects with `__get__/__set__/__delete__`; power `property`, methods, `classmethod`, and Django model fields.

```
class Positive:
    def __set_name__(self, owner, name): self.name = "_" + name
    def __get__(self, obj, objtype=None): return getattr(obj, self.name)
    def __set__(self, obj, value):
        if value <= 0: raise ValueError(f"{self.name[1:]} must be > 0")
        setattr(obj, self.name, value)

class Order:
    qty = Positive()
    def __init__(self, qty): self.qty = qty
```

5.14 Metaclasses (Advanced)

A metaclass is the "class of a class" — `type` by default. Used by Django ORM (`ModelBase`) to turn field declarations into a model.

```
class RegistryMeta(type):
    registry = {}
    def __new__(mcls, name, bases, ns):
        cls = super().__new__(mcls, name, bases, ns)
        if bases:
```

```
            mcls.registry[name.lower()] = cls
        return cls

class Plugin(metaclass=RegistryMeta): ...
class CsvExporter(Plugin): ...
RegistryMeta.registry   # {'csvexporter': <class CsvExporter>}

# Often simpler: __init_subclass__
class Base:
    plugins = []
    def __init_subclass__(cls, **kw):
        super().__init_subclass__(**kw)
        Base.plugins.append(cls)
```

5.15 Testing with pytest

```
# test_cart.py
import pytest
from cart import Cart

@pytest.fixture
def cart():
    return Cart()

def test_add_item(cart):
    cart.add("apple", 2)
    assert cart.total_items == 2

@pytest.mark.parametrize("qty, expected", [(1, 10), (3, 30), (0, 0)])
def test_price(cart, qty, expected):
    cart.add("x", qty, price=10)
    assert cart.total == expected

def test_negative_qty_raises(cart):
    with pytest.raises(ValueError, match="positive"):
        cart.add("x", -1)

def test_external_api(mocker):                    # pytest-mock
    mock_get = mocker.patch("cart.requests.get")
    mock_get.return_value.json.return_value = {"rate": 1.1}
    assert Cart().convert(10) == 11

# unittest.mock directly
from unittest.mock import patch, MagicMock
with patch("module.send_email") as fake:
    do_signup()
    fake.assert_called_once_with("a@b.com")
```

5.16 Common Interview Snippets

```
# Reverse words
" ".join(reversed("hello big world".split()))

# Anagram check
from collections import Counter
Counter("listen") == Counter("silent")

# Two sum — O(n)
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i
```

```python
# Flatten nested dict
def flatten(d, parent=""):
    out = {}
    for k, v in d.items():
        key = f"{parent}.{k}" if parent else k
        out.update(flatten(v, key) if isinstance(v, dict) else {key: v})
    return out

# LRU cache implementation
from collections import OrderedDict
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)

# Singleton
class Singleton:
    _instance = None
    def __new__(cls, *a, **kw):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Tricky outputs:

```python
print([[0] * 2] * 2)        # [[0,0],[0,0]] but rows share the same list!
x = [[0] * 2] * 2; x[0][0] = 1; print(x)   # [[1,0],[1,0]] — use [[0]*2 for _ in range(2)]
print(0.1 + 0.2 == 0.3)     # False → math.isclose
print(bool([]), bool([0]))  # False True
print(round(2.5), round(3.5))  # 2 4 — banker's rounding
fs = [lambda: i for i in range(3)]; print([f() for f in fs])  # [2,2,2] — late binding
fs = [lambda i=i: i for i in range(3)]                        # [0,1,2]
```

Part 6 — Django & Django REST Framework

6.1 Architecture: MTV and the Request Lifecycle

Django follows **MTV** — **M**odel (data), **T**emplate (presentation), **V**iew (business logic; the "controller" role is played by the framework + URLconf).

Request lifecycle:

1. WSGI/ASGI server (Gunicorn / Uvicorn) receives the request → `WSGIHandler` / `ASGIHandler`.
2. **Middleware** (request phase, top → bottom of `MIDDLEWARE`).
3. **URL resolver** matches `urls.py` patterns → view.
4. **View** runs (talks to models/ORM, services, cache).
5. Returns `HttpResponse` (template render or JSON).
6. **Middleware** (response phase, bottom → top).
7. Response sent to client.

```
myproject/
  manage.py
  config/            # settings.py, urls.py, wsgi.py, asgi.py
  apps/
    orders/
      models.py  views.py  serializers.py  urls.py  admin.py
      services.py  selectors.py  signals.py  tasks.py  tests/
      migrations/
```

```
django-admin startproject config .
python manage.py startapp orders
python manage.py makemigrations && python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
python manage.py shell_plus --print-sql   # django-extensions
```

6.2 Settings Essentials

```python
# settings.py
import os
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.getenv("DEBUG") == "1"
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "").split(",")

INSTALLED_APPS = [
    "django.contrib.admin", "django.contrib.auth", "django.contrib.contenttypes",
    "django.contrib.sessions", "django.contrib.messages", "django.contrib.staticfiles",
    "rest_framework", "django_filters", "corsheaders",
    "apps.orders",
]

DATABASES = {
```

```
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"], "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"], "HOST": os.environ["DB_HOST"], "PORT": 5432,
        "CONN_MAX_AGE": 60,          # persistent connections
        "ATOMIC_REQUESTS": False,
    }
}

AUTH_USER_MODEL = "users.User"      # set a custom user model at project start!
```

Split settings per environment (`base.py` , `dev.py` , `prod.py`) or use `django-environ` .

6.3 Models

```
from django.db import models
from django.conf import settings
from django.core.validators import MinValueValidator

class TimeStamped(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True                         # no table; fields inherited


class Category(TimeStamped):
    name = models.CharField(max_length=100, unique=True)
    slug = models.SlugField(unique=True)

    def __str__(self):
        return self.name


class ProductQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)
    def in_stock(self):
        return self.filter(stock__gt=0)


class Product(TimeStamped):
    class Status(models.TextChoices):
        DRAFT = "draft", "Draft"
        PUBLISHED = "published", "Published"

    category = models.ForeignKey(Category, on_delete=models.PROTECT, related_name="products")
    name = models.CharField(max_length=200, db_index=True)
    price = models.DecimalField(max_digits=10, decimal_places=2, validators=
[MinValueValidator(0)])
    stock = models.PositiveIntegerField(default=0)
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.DRAFT)
    is_active = models.BooleanField(default=True)
    tags = models.ManyToManyField("Tag", blank=True, related_name="products")
    metadata = models.JSONField(default=dict, blank=True)

    objects = ProductQuerySet.as_manager()      # Product.objects.active().in_stock()

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["category", "status"])]
        constraints = [
            models.CheckConstraint(condition=models.Q(price__gte=0), name="price_non_negative"),
            models.UniqueConstraint(fields=["category", "name"],
name="uniq_product_per_category"),
        ]
```

```
    def __str__(self):
        return self.name


class Tag(models.Model):
    name = models.CharField(max_length=50, unique=True)


class Order(TimeStamped):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="orders")
    products = models.ManyToManyField(Product, through="OrderItem")   # M2M with extra data
    total = models.DecimalField(max_digits=12, decimal_places=2, default=0)


class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name="items")
    product = models.ForeignKey(Product, on_delete=models.PROTECT)
    quantity = models.PositiveIntegerField()
    unit_price = models.DecimalField(max_digits=10, decimal_places=2)


class Profile(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    bio = models.TextField(blank=True)
```

`on_delete` options: `CASCADE` (delete children), `PROTECT` (block deletion), `RESTRICT`, `SET_NULL` (needs `null=True`), `SET_DEFAULT`, `SET(...)`, `DO_NOTHING`.

`null` vs `blank`: `null=True` → DB column allows NULL; `blank=True` → form/validation allows empty. For strings prefer `blank=True` with `""` instead of `null=True`.

Model inheritance

Type	Tables	Use
Abstract base (`abstract = True`)	Only children	Shared fields (timestamps)
Multi-table	Parent + child joined 1:1	Real "is-a" with queries on parent
Proxy (`proxy = True`)	Same table	Different Python behaviour / manager / admin

6.4 ORM — QuerySets

QuerySets are **lazy** — no SQL runs until evaluated (iteration, `list()`, slicing with step, `len()`, `bool()`, `repr()`). They are also **cached** after evaluation.

```
from django.db.models import Q, F, Count, Sum, Avg, Max, Value, Case, When, Subquery, OuterRef,
Exists
from django.db.models.functions import Coalesce, Lower, TruncMonth

Product.objects.all()
Product.objects.filter(price__gte=100, category__slug="phones")   # JOIN via __
Product.objects.exclude(status="draft")
Product.objects.get(pk=1)                    # DoesNotExist / MultipleObjectsReturned
Product.objects.filter(name__icontains="pro").order_by("-price")[:10]  # LIMIT 10
Product.objects.values("id", "name")         # dicts
Product.objects.values_list("id", flat=True) # [1, 2, ...]
```

```python
Product.objects.only("id", "name")           # defer other columns
Product.objects.exists(); Product.objects.count()
obj, created = Product.objects.get_or_create(name="X", defaults={"price": 1, "category": c})
Product.objects.update_or_create(name="X", defaults={"price": 2})

# Field lookups
# exact, iexact, contains, icontains, in, gt, gte, lt, lte, startswith, range,
# isnull, date, year, month, regex

# Q objects: OR / NOT
Product.objects.filter(Q(price__lt=10) | Q(stock=0), ~Q(status="draft"))

# F expressions: reference columns, atomic updates (avoid race conditions)
Product.objects.filter(stock__lt=F("reorder_level"))
Product.objects.filter(pk=1).update(stock=F("stock") - 1)

# Aggregation (whole table) vs annotation (per row)
Product.objects.aggregate(avg=Avg("price"), total=Sum("stock"))
Category.objects.annotate(n=Count("products")).filter(n__gt=5).order_by("-n")

# Group by month
(Order.objects.annotate(month=TruncMonth("created_at"))
     .values("month").annotate(revenue=Sum("total")).order_by("month"))

# Conditional expressions
Product.objects.annotate(
    tier=Case(When(price__gte=1000, then=Value("premium")), default=Value("standard"))
)

# Subqueries
latest = Order.objects.filter(user=OuterRef("pk")).order_by("-created_at")
User.objects.annotate(
    last_order_total=Subquery(latest.values("total")[:1]),
    has_orders=Exists(Order.objects.filter(user=OuterRef("pk"))),
)

# Bulk operations — far fewer queries
Product.objects.bulk_create([Product(name=f"p{i}", price=1, category=c) for i in range(1000)],
batch_size=500)
Product.objects.bulk_update(products, ["price"], batch_size=500)
Product.objects.filter(status="draft").delete()

# Raw SQL when needed (still parameterized)
Product.objects.raw("SELECT * FROM shop_product WHERE price > %s", [100])
from django.db import connection
with connection.cursor() as cur:
    cur.execute("SELECT count(*) FROM shop_product WHERE stock = %s", [0])
    cur.fetchone()

print(qs.query)            # see SQL
qs.explain(analyze=True)   # EXPLAIN on Postgres
```

The N+1 Problem: `select_related` vs `prefetch_related`

```python
# N+1: 1 query for orders + 1 query per order for user
for o in Order.objects.all():
    print(o.user.email)

# select_related → SQL JOIN — for ForeignKey / OneToOne (forward)
for o in Order.objects.select_related("user"):
    print(o.user.email)                # 1 query total

# prefetch_related → separate query + Python join — for M2M & reverse FK
for o in Order.objects.prefetch_related("items__product"):
    for item in o.items.all():         # 3 queries total, regardless of N
        print(item.product.name)

# Prefetch object for filtered/optimized prefetches
from django.db.models import Prefetch
```

```
Order.objects.prefetch_related(
    Prefetch("items",
queryset=OrderItem.objects.select_related("product").filter(quantity__gt=1),
             to_attr="big_items")
)
```

Detect N+1 with `django-debug-toolbar`, `nplusone`, or `assertNumQueries` in tests.

6.5 Transactions and Concurrency

```
from django.db import transaction

@transaction.atomic
def place_order(user, cart):
    # SELECT ... FOR UPDATE — lock rows until the transaction ends
    products = Product.objects.select_for_update().filter(id__in=cart.keys())
    order = Order.objects.create(user=user)
    for p in products:
        qty = cart[p.id]
        if p.stock < qty:
            raise ValueError(f"{p.name} out of stock")    # rolls back everything
        p.stock = F("stock") - qty
        p.save(update_fields=["stock"])
        OrderItem.objects.create(order=order, product=p, quantity=qty, unit_price=p.price)

    # Run side effects only after a successful commit
    transaction.on_commit(lambda: send_order_email.delay(order.id))
    return order

# Savepoints with nested atomic blocks
with transaction.atomic():
    create_user()
    try:
        with transaction.atomic():      # savepoint
            risky()
    except IntegrityError:
        pass                            # outer transaction still alive
```

Race-condition tools: `F()` expressions, `select_for_update(nowait=True / skip_locked=True)`, unique constraints + `IntegrityError` handling, optimistic locking with a `version` column.

6.6 Migrations

```
python manage.py makemigrations orders
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate orders 0003      # show SQL
python manage.py migrate orders 0002         # roll back
python manage.py squashmigrations orders 0001 0010
```

```
# Data migration
from django.db import migrations

def forwards(apps, schema_editor):
    Product = apps.get_model("shop", "Product")       # use historical model!
    Product.objects.filter(status="").update(status="draft")

class Migration(migrations.Migration):
    dependencies = [("shop", "0004_product_status")]
    operations = [migrations.RunPython(forwards, migrations.RunPython.noop)]
```

Zero-downtime tips: add nullable columns first, backfill in batches, then add NOT NULL; create indexes concurrently on Postgres (`AddIndexConcurrently`, `atomic = False`); never rename/drop columns used by running code in the same deploy.

6.7 Views

Function-Based Views

```python
from django.shortcuts import render, get_object_or_404, redirect
from django.http import JsonResponse
from django.contrib.auth.decorators import login_required
from django.views.decorators.http import require_http_methods

@login_required
@require_http_methods(["GET", "POST"])
def product_detail(request, pk):
    product = get_object_or_404(Product, pk=pk)
    if request.method == "POST":
        form = ProductForm(request.POST, instance=product)
        if form.is_valid():
            form.save()
            return redirect("product-detail", pk=pk)
    else:
        form = ProductForm(instance=product)
    return render(request, "shop/detail.html", {"product": product, "form": form})
```

Class-Based Views

```python
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView
from django.contrib.auth.mixins import LoginRequiredMixin, PermissionRequiredMixin
from django.urls import reverse_lazy

class ProductListView(LoginRequiredMixin, ListView):
    model = Product
    paginate_by = 20
    template_name = "shop/list.html"
    context_object_name = "products"

    def get_queryset(self):
        qs = super().get_queryset().select_related("category")
        if q := self.request.GET.get("q"):
            qs = qs.filter(name__icontains=q)
        return qs

class ProductCreateView(PermissionRequiredMixin, CreateView):
    model = Product
    fields = ["name", "price", "category"]
    permission_required = "shop.add_product"
    success_url = reverse_lazy("product-list")

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

CBV flow: `as_view()` → `dispatch()` → `get()/post()` → `get_queryset()` / `get_context_data()` → `render_to_response()`.

URLs

```python
from django.urls import path, include
urlpatterns = [
    path("products/", ProductListView.as_view(), name="product-list"),
    path("products/<int:pk>/", product_detail, name="product-detail"),
    path("blog/<slug:slug>/", post_detail),
    path("api/", include("apps.api.urls")),
]
# reverse("product-detail", kwargs={"pk": 1}); in templates {% url 'product-detail' pk=1 %}
```

6.8 Templates and Forms (Quick)

```
{% extends "base.html" %}
{% load static %}
{% block content %}
  {% for p in products %}
    <h3>{{ p.name|title }}</h3> <p>{{ p.price|floatformat:2 }}</p>
  {% empty %}
    <p>No products.</p>
  {% endfor %}
  <form method="post">{% csrf_token %}{{ form.as_p }}<button>Save</button></form>
{% endblock %}
```

```python
from django import forms
class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "price", "category"]
    def clean_price(self):
        price = self.cleaned_data["price"]
        if price > 1_000_000:
            raise forms.ValidationError("Too expensive")
        return price
    def clean(self):                        # cross-field validation
        data = super().clean()
        return data
```

Templates auto-escape HTML (XSS protection); `|safe` disables it — use carefully.

6.9 Middleware

```python
import time, logging, uuid
logger = logging.getLogger(__name__)

class RequestTimingMiddleware:
    def __init__(self, get_response):          # once at startup
        self.get_response = get_response

    def __call__(self, request):               # every request
        request.request_id = str(uuid.uuid4())
        start = time.monotonic()
        response = self.get_response(request)  # → next middleware / view
        response["X-Request-ID"] = request.request_id
        logger.info("%s %s %.1fms", request.method, request.path, (time.monotonic() - start) *
1000)
        return response

    def process_exception(self, request, exception):
```

```
        logger.exception("unhandled error id=%s", request.request_id)
        return None                             # let Django handle it
```

Other hooks: `process_view`, `process_template_response`. Order in `MIDDLEWARE` matters (e.g., `SecurityMiddleware` first, `SessionMiddleware` before `AuthenticationMiddleware`, `CorsMiddleware` high up).

6.10 Signals

```
from django.db.models.signals import post_save, pre_delete
from django.dispatch import receiver

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
# Register in AppConfig.ready(): import apps.users.signals
```

Common signals: `pre_save`, `post_save`, `pre_delete`, `post_delete`, `m2m_changed`, `request_started`, `user_logged_in`. **Caution:** signals hide control flow, are not fired by `bulk_create` / `update()` / `QuerySet.delete()` (for pre/post_save) — prefer explicit service functions for business logic.

6.11 Authentication and Permissions

```
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    email = models.EmailField(unique=True)
    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["username"]

# Usage
from django.contrib.auth import authenticate, login, logout
user = authenticate(request, username=email, password=pw)
if user: login(request, user)

user.has_perm("shop.change_product")
user.groups.add(Group.objects.get(name="editors"))
```

Passwords are hashed with PBKDF2 by default (Argon2 supported). Sessions stored in DB/cache; CSRF middleware protects cookie-authenticated POSTs.

6.12 Django REST Framework (DRF)

Serializers

```
from rest_framework import serializers

class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ["id", "name", "slug"]
```

```python
class ProductSerializer(serializers.ModelSerializer):
    category = CategorySerializer(read_only=True)
    category_id = serializers.PrimaryKeyRelatedField(
        queryset=Category.objects.all(), source="category", write_only=True
    )
    discounted_price = serializers.SerializerMethodField()

    class Meta:
        model = Product
        fields = ["id", "name", "price", "stock", "category", "category_id", "discounted_price"]
        read_only_fields = ["stock"]

    def get_discounted_price(self, obj):
        return obj.price * 90 / 100

    def validate_price(self, value):                    # field-level
        if value <= 0:
            raise serializers.ValidationError("Price must be positive")
        return value

    def validate(self, attrs):                          # object-level
        if attrs.get("name", "").lower() == "test":
            raise serializers.ValidationError({"name": "Reserved name"})
        return attrs


class OrderCreateSerializer(serializers.Serializer):    # nested writes
    items = serializers.ListField(child=serializers.DictField(), min_length=1)

    def create(self, validated_data):
        return place_order(self.context["request"].user, validated_data["items"])
```

Flow: `serializer = S(data=request.data)` → `is_valid(raise_exception=True)` → `validated_data` → `save()` calls `create()` or `update()` → `serializer.data` for output.

Views, ViewSets, Routers

```python
from rest_framework import viewsets, generics, status, permissions, filters
from rest_framework.decorators import action, api_view, permission_classes
from rest_framework.response import Response
from django_filters.rest_framework import DjangoFilterBackend

@api_view(["GET"])
@permission_classes([permissions.AllowAny])
def health(request):
    return Response({"ok": True})


class ProductViewSet(viewsets.ModelViewSet):
    serializer_class = ProductSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
    filterset_fields = ["category", "status"]
    search_fields = ["name", "category__name"]
    ordering_fields = ["price", "created_at"]

    def get_queryset(self):
        return Product.objects.active().select_related("category").prefetch_related("tags")

    def get_serializer_class(self):
        return ProductListSerializer if self.action == "list" else ProductSerializer

    def perform_create(self, serializer):
        serializer.save(created_by=self.request.user)

    @action(detail=True, methods=["post"], url_path="restock")
    def restock(self, request, pk=None):
        product = self.get_object()
```

```
        qty = int(request.data.get("quantity", 0))
        Product.objects.filter(pk=product.pk).update(stock=F("stock") + qty)
        return Response({"status": "restocked"}, status=status.HTTP_200_OK)


# urls.py
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register("products", ProductViewSet, basename="product")
urlpatterns = router.urls
# GET/POST /products/, GET/PUT/PATCH/DELETE /products/{id}/, POST /products/{id}/restock/
```

Hierarchy: `APIView` → `GenericAPIView` (+ mixins: `ListModelMixin`, `CreateModelMixin`, ...) → concrete generics (`ListCreateAPIView`, `RetrieveUpdateDestroyAPIView`) → `ViewSet` / `ModelViewSet`.

Authentication, Permissions, Throttling, Pagination

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework_simplejwt.authentication.JWTAuthentication",
        "rest_framework.authentication.SessionAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": ["rest_framework.permissions.IsAuthenticated"],
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 20,
    "DEFAULT_THROTTLE_CLASSES": ["rest_framework.throttling.UserRateThrottle",
                                 "rest_framework.throttling.AnonRateThrottle"],
    "DEFAULT_THROTTLE_RATES": {"user": "1000/day", "anon": "100/day"},
    "EXCEPTION_HANDLER": "apps.core.exceptions.custom_exception_handler",
}

# Custom object-level permission
from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsOwnerOrReadOnly(BasePermission):
    def has_permission(self, request, view):            # view-level
        return request.user.is_authenticated
    def has_object_permission(self, request, view, obj): # called by get_object()
        return request.method in SAFE_METHODS or obj.owner_id == request.user.id

# Custom exception handler
from rest_framework.views import exception_handler
def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)
    if response is not None:
        response.data = {"error": {"status": response.status_code, "detail": response.data}}
    return response
```

Pagination options: `PageNumberPagination`, `LimitOffsetPagination`, `CursorPagination` (stable & efficient for large/real-time datasets).

6.13 Caching

```
CACHES = {"default": {"BACKEND": "django.core.cache.backends.redis.RedisCache",
                      "LOCATION": "redis://127.0.0.1:6379/1"}}

from django.core.cache import cache
from django.views.decorators.cache import cache_page

def get_dashboard(user_id):
    key = f"dashboard:{user_id}"
    return cache.get_or_set(key, lambda: build_dashboard(user_id), timeout=300)
```

```
cache.delete("dashboard:1")

@cache_page(60 * 15)                # per-view cache
def product_list(request): ...
# Template fragment cache: {% load cache %}{% cache 500 sidebar %}...{% endcache %}
```

6.14 Celery (Background Tasks)

```
# config/celery.py
import os
from celery import Celery
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
app = Celery("config")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()

# apps/orders/tasks.py
from celery import shared_task

@shared_task(bind=True, autoretry_for=(ConnectionError,), retry_backoff=True, max_retries=5)
def send_order_email(self, order_id):
    order = Order.objects.select_related("user").get(pk=order_id)   # pass IDs, not objects
    send_mail("Order placed", f"#{order.id}", None, [order.user.email])

# Periodic tasks (celery beat)
CELERY_BEAT_SCHEDULE = {
    "nightly-report": {"task": "apps.reports.tasks.nightly", "schedule": crontab(hour=2,
minute=0)},
}
```

```
celery -A config worker -l info --concurrency=4
celery -A config beat -l info
```

Best practices: idempotent tasks, pass primitive IDs, call `.delay()` inside `transaction.on_commit`, set time limits, use separate queues for slow/fast tasks.

6.15 Async Django

Django supports async views (3.1+), async ORM methods (`aget`, `acreate`, `afilter` iteration via `async for`) and ASGI deployment.

```
from django.http import JsonResponse
import httpx

async def weather(request):
    async with httpx.AsyncClient() as client:
        r = await client.get("https://api.example.com/weather")
    product = await Product.objects.aget(pk=1)
    count = await Product.objects.filter(stock=0).acount()
    return JsonResponse({"weather": r.json(), "out_of_stock": count})

from asgiref.sync import sync_to_async
result = await sync_to_async(legacy_sync_function)()
```

Real-time features (WebSockets) use **Django Channels**.

6.16 Testing Django

```python
import pytest
from django.urls import reverse
from rest_framework.test import APIClient

@pytest.fixture
def api_client(django_user_model):
    user = django_user_model.objects.create_user(username="a", email="a@x.com", password="pw")
    client = APIClient()
    client.force_authenticate(user)
    return client

@pytest.mark.django_db
def test_create_product(api_client, category):
    resp = api_client.post(reverse("product-list"),
                           {"name": "Phone", "price": "999.00", "category_id": category.id},
                           format="json")
    assert resp.status_code == 201
    assert Product.objects.filter(name="Phone").exists()

@pytest.mark.django_db
def test_list_has_no_n_plus_one(api_client, django_assert_num_queries, products):
    with django_assert_num_queries(3):
        api_client.get(reverse("product-list"))
```

Tools: `pytest-django`, `factory_boy` (test data), `freezegun` (time), `responses` / `respx` (mock HTTP), `TestCase` (wraps each test in a transaction), `TransactionTestCase` (for on_commit/real transactions), `override_settings`.

6.17 Security Checklist

- `DEBUG = False`, strong `SECRET_KEY` from env, correct `ALLOWED_HOSTS`.
- `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS`.
- Built-in protections: CSRF tokens, template auto-escaping (XSS), ORM parameterization (SQL injection), clickjacking (`X-Frame-Options`), password hashing & validators.
- Never use `extra()` / raw SQL with string formatting.
- Validate uploads; serve media from a separate domain/bucket.
- `python manage.py check --deploy`.

6.18 Performance and Deployment

- Fix N+1 (`select_related` / `prefetch_related`), use `only()/defer()`, `values()` for read-only data, `iterator(chunk_size=…)` for huge loops, `bulk_*` for writes, `exists()` instead of `count() > 0`.
- Add DB indexes for filter/order columns; analyse with `EXPLAIN`.
- Cache aggressively (Redis), use CDN for static files (`collectstatic`, WhiteNoise/S3).
- Run with Gunicorn (`gunicorn config.wsgi -w 4 -k gthread`) or Uvicorn for ASGI, behind Nginx.
- Connection pooling (`CONN_MAX_AGE`, PgBouncer, Django 5.1 native pool option for psycopg3).
- Offload slow work to Celery; monitor with Sentry, Prometheus, APM.

6.19 Service Layer Pattern (Clean Django)

```python
# services.py — writes / business logic
def product_create(*, name: str, price: Decimal, category: Category, user) -> Product:
    product = Product(name=name, price=price, category=category, created_by=user)
    product.full_clean()
    product.save()
    audit_log(user, "product_created", product.id)
    return product

# selectors.py — reads
def product_list(*, filters: dict | None = None) -> QuerySet[Product]:
    qs = Product.objects.active().select_related("category")
    return ProductFilter(filters or {}, qs).qs

# Views stay thin: validate input → call service/selector → serialize output
```

6.20 Interview Questions Checklist

- Explain the request/response lifecycle and middleware order.
- `select_related` vs `prefetch_related`; how to detect N+1.
- QuerySet laziness and caching; when is a query executed?
- `F()` and `Q()` objects; `annotate` vs `aggregate`.
- How `transaction.atomic` and `select_for_update` work; `on_commit`.
- `null` vs `blank`; `on_delete` options; custom user model.
- Signals: pros/cons; why they don't fire on `bulk_create` / `update`.
- CBV vs FBV; mixins and MRO.
- DRF: serializer validation flow, ViewSets vs APIView, permissions (`has_permission` vs `has_object_permission`), throttling, pagination.
- Celery: idempotency, retries, beat; why pass IDs.
- WSGI vs ASGI; async views.
- How Django protects against CSRF, XSS, SQL injection, clickjacking.

Part 7 — SQL: Basics to Advanced

Examples use **PostgreSQL** syntax (differences with MySQL noted where important). Sample schema used throughout:

```sql
CREATE TABLE departments (
    id   SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE employees (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(255) UNIQUE NOT NULL,
    department_id INT REFERENCES departments(id) ON DELETE SET NULL,
    manager_id    INT REFERENCES employees(id),
    salary        NUMERIC(10,2) CHECK (salary > 0),
    hired_at      DATE NOT NULL DEFAULT CURRENT_DATE,
    is_active     BOOLEAN DEFAULT TRUE
);

CREATE TABLE orders (
    id          BIGSERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    amount      NUMERIC(12,2) NOT NULL,
    status      VARCHAR(20) NOT NULL DEFAULT 'pending',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

7.1 SQL Sub-languages

Category	Commands
DDL (Definition)	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`
DML (Manipulation)	`INSERT`, `UPDATE`, `DELETE`, `MERGE`
DQL (Query)	`SELECT`
DCL (Control)	`GRANT`, `REVOKE`
TCL (Transaction)	`BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`

7.2 Data Types (Common)

- Numeric: `SMALLINT`, `INT`, `BIGINT`, `NUMERIC(p,s)` / `DECIMAL` (exact — use for money), `REAL` / `DOUBLE PRECISION` (approximate).
- Text: `CHAR(n)` (fixed), `VARCHAR(n)`, `TEXT`.
- Date/time: `DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMPTZ` (store UTC — preferred), `INTERVAL`.
- Others: `BOOLEAN`, `UUID`, `JSON` / `JSONB` (binary, indexable), `ARRAY`, `ENUM`, `BYTEA`.

7.3 DDL and Constraints

```sql
ALTER TABLE employees ADD COLUMN phone VARCHAR(20);
ALTER TABLE employees ALTER COLUMN phone SET NOT NULL;
ALTER TABLE employees DROP COLUMN phone;
ALTER TABLE employees RENAME COLUMN name TO full_name;
ALTER TABLE employees ADD CONSTRAINT chk_email CHECK (email LIKE '%@%');

DROP TABLE IF EXISTS temp_data;          -- removes table + structure
TRUNCATE TABLE logs RESTART IDENTITY;    -- removes all rows fast, keeps structure
```

Constraints: `PRIMARY KEY` (unique + not null), `FOREIGN KEY` (referential integrity; `ON DELETE CASCADE | SET NULL | RESTRICT`), `UNIQUE`, `NOT NULL`, `CHECK`, `DEFAULT`.

DELETE vs TRUNCATE vs DROP:

	DELETE	TRUNCATE	DROP
Removes	Selected rows (`WHERE`)	All rows	Whole table
Speed	Slower (row by row, logs each)	Fast	Fast
Triggers fire	Yes	No	No
Rollback	Yes	Yes in Postgres (not in MySQL)	Yes in Postgres

7.4 Basic DML and Queries

```sql
INSERT INTO departments (name) VALUES ('Engineering'), ('Sales'), ('HR');

INSERT INTO employees (name, email, department_id, salary)
VALUES ('Anish', 'anish@x.com', 1, 120000)
RETURNING id;                                -- Postgres

-- Upsert
INSERT INTO employees (email, name, salary) VALUES ('a@x.com', 'A', 50000)
ON CONFLICT (email) DO UPDATE SET salary = EXCLUDED.salary;
-- MySQL: INSERT ... ON DUPLICATE KEY UPDATE salary = VALUES(salary);

UPDATE employees SET salary = salary * 1.10 WHERE department_id = 1;
DELETE FROM employees WHERE is_active = FALSE;

SELECT DISTINCT department_id FROM employees;

SELECT name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 150000
  AND name ILIKE 'a%'                -- LIKE: % any chars, _ single char; ILIKE = case-
insensitive
  AND department_id IN (1, 2)
  AND manager_id IS NOT NULL         -- never "= NULL"
ORDER BY salary DESC NULLS LAST, name
LIMIT 10 OFFSET 20;                  -- pagination (SQL Server: OFFSET ... FETCH NEXT)

SELECT name,
       CASE
         WHEN salary >= 150000 THEN 'High'
         WHEN salary >= 80000  THEN 'Mid'
         ELSE 'Low'
       END AS band,
       COALESCE(manager_id, 0) AS mgr,   -- first non-null
```

```
       NULLIF(salary, 0)                -- NULL if equal
FROM employees;
```

NULL semantics (three-valued logic)

- `NULL = NULL` → `NULL` (unknown), not true. Use `IS NULL` / `IS DISTINCT FROM`.
- Aggregates ignore NULLs (except `COUNT(*)`).
- `NOT IN (subquery)` returns no rows if the subquery contains a NULL → prefer `NOT EXISTS`.

7.5 Logical Order of Execution

```
FROM / JOIN  →  WHERE  →  GROUP BY  →  HAVING  →  SELECT  →  DISTINCT  →  ORDER BY  →
LIMIT/OFFSET
```

That is why you cannot use a `SELECT` alias inside `WHERE` (it doesn't exist yet), but you can in `ORDER BY`. Window functions are computed after `HAVING`, before `ORDER BY`.

7.6 Aggregation: GROUP BY and HAVING

```
SELECT department_id,
       COUNT(*)            AS headcount,
       COUNT(manager_id)   AS with_manager,   -- counts non-null
       AVG(salary)::INT    AS avg_salary,
       MIN(salary), MAX(salary), SUM(salary),
       STRING_AGG(name, ', ' ORDER BY name) AS names  -- MySQL: GROUP_CONCAT
FROM employees
WHERE is_active
GROUP BY department_id
HAVING COUNT(*) > 5             -- filter groups (WHERE filters rows)
ORDER BY avg_salary DESC;

-- Conditional aggregation (pivot)
SELECT DATE_TRUNC('month', created_at) AS month,
       COUNT(*) FILTER (WHERE status = 'paid')     AS paid,
       COUNT(*) FILTER (WHERE status = 'refunded') AS refunded,
       SUM(CASE WHEN status = 'paid' THEN amount ELSE 0 END) AS revenue
FROM orders
GROUP BY 1
ORDER BY 1;

-- Subtotals
SELECT department_id, status, SUM(amount)
FROM orders GROUP BY ROLLUP (department_id, status);  -- also CUBE, GROUPING SETS
```

7.7 Joins

```
-- INNER: only matching rows
SELECT e.name, d.name AS dept
FROM employees e
JOIN departments d ON d.id = e.department_id;

-- LEFT: all rows from left, NULLs when no match
SELECT d.name, COUNT(e.id) AS headcount        -- COUNT(e.id) not COUNT(*) → 0 for empty depts
FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
GROUP BY d.name;
```

```sql
-- Anti-join: departments with NO employees
SELECT d.* FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
WHERE e.id IS NULL;

-- RIGHT JOIN (mirror of LEFT), FULL OUTER JOIN (all rows from both)
SELECT * FROM a FULL OUTER JOIN b ON a.id = b.a_id;

-- CROSS JOIN: Cartesian product (every combination)
SELECT s.size, c.color FROM sizes s CROSS JOIN colors c;

-- SELF JOIN: employee with manager name
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id;

-- Join with extra condition: put it in ON (not WHERE) for LEFT joins to keep unmatched rows
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'paid';
```

Join algorithms (chosen by the planner): Nested Loop (small inner/indexed), Hash Join (large unsorted equi-joins), Merge Join (both inputs sorted).

7.8 Set Operations

```sql
SELECT email FROM customers
UNION                -- removes duplicates (sort/hash cost)
SELECT email FROM newsletter;

SELECT email FROM customers UNION ALL SELECT email FROM newsletter;  -- keeps dupes, faster
SELECT id FROM a INTERSECT SELECT id FROM b;
SELECT id FROM a EXCEPT SELECT id FROM b;       -- Oracle: MINUS
```

Column count and compatible types must match.

7.9 Subqueries

```sql
-- Scalar subquery
SELECT name, salary, (SELECT AVG(salary) FROM employees) AS company_avg FROM employees;

-- IN subquery
SELECT * FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE name IN ('Engineering', 'Sales'));

-- Correlated subquery: runs per outer row — employees earning above their dept average
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id);

-- EXISTS / NOT EXISTS (stops at first match; NULL-safe)
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- Derived table
SELECT dept, avg_sal FROM (
    SELECT department_id AS dept, AVG(salary) AS avg_sal FROM employees GROUP BY department_id
) t WHERE avg_sal > 100000;

-- LATERAL: top 3 orders per customer
SELECT c.id, o.*
FROM customers c
```

```sql
CROSS JOIN LATERAL (
    SELECT * FROM orders WHERE customer_id = c.id ORDER BY amount DESC LIMIT 3
) o;
```

7.10 Common Table Expressions (CTEs)

```sql
WITH dept_stats AS (
    SELECT department_id, AVG(salary) AS avg_sal
    FROM employees GROUP BY department_id
),
high_paid AS (
    SELECT e.* FROM employees e
    JOIN dept_stats s USING (department_id)
    WHERE e.salary > s.avg_sal * 1.2
)
SELECT * FROM high_paid;
```

Recursive CTE — hierarchy / org chart

```sql
WITH RECURSIVE org AS (
    SELECT id, name, manager_id, 1 AS level, name::TEXT AS path
    FROM employees WHERE manager_id IS NULL                -- anchor: CEO
    UNION ALL
    SELECT e.id, e.name, e.manager_id, o.level + 1, o.path || ' > ' || e.name
    FROM employees e
    JOIN org o ON e.manager_id = o.id                       -- recursive step
)
SELECT * FROM org ORDER BY path;

-- Generate a date series (useful to fill gaps in reports)
WITH RECURSIVE days AS (
    SELECT DATE '2026-01-01' AS d
    UNION ALL SELECT d + 1 FROM days WHERE d < DATE '2026-01-31'
)
SELECT d FROM days;
-- Postgres shortcut: SELECT generate_series('2026-01-01'::date, '2026-01-31', '1 day');
```

7.11 Window Functions

Window functions compute a value **per row** over a set of related rows **without collapsing** them (unlike `GROUP BY`).

```sql
function() OVER (PARTITION BY ... ORDER BY ... ROWS/RANGE BETWEEN ... )
```

```sql
-- Ranking
SELECT name, department_id, salary,
       ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn,   -- 1,2,3,4
       RANK()       OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk,  -- 1,2,2,4
       DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS drnk, -- 1,2,2,3
       NTILE(4)     OVER (ORDER BY salary) AS quartile,
       PERCENT_RANK() OVER (ORDER BY salary) AS pct
FROM employees;

-- Top-N per group (classic interview question)
SELECT * FROM (
    SELECT e.*, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS r
    FROM employees e
) t WHERE r <= 3;
```

```sql
-- Aggregates as windows
SELECT name, department_id, salary,
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY department_id) AS diff_from_avg,
       SUM(salary) OVER () AS company_total,
       ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY department_id), 2) AS pct_of_dept
FROM employees;

-- Running total & moving average
SELECT created_at::date AS day, amount,
       SUM(amount) OVER (ORDER BY created_at ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AS running_total,
       AVG(amount) OVER (ORDER BY created_at ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
moving_avg_7
FROM orders;

-- LAG / LEAD: compare with previous / next row
SELECT month, revenue,
       LAG(revenue)  OVER (ORDER BY month) AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS mom_pct,
       LEAD(revenue) OVER (ORDER BY month) AS next_month
FROM monthly_revenue;

-- FIRST_VALUE / LAST_VALUE / NTH_VALUE
SELECT name, department_id,
       FIRST_VALUE(name) OVER w AS top_earner,
       LAST_VALUE(name)  OVER (w ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
lowest
FROM employees
WINDOW w AS (PARTITION BY department_id ORDER BY salary DESC);
```

ROWS vs RANGE: `ROWS` counts physical rows; `RANGE` groups peers with equal `ORDER BY` values. The default frame with `ORDER BY` is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` — the reason `LAST_VALUE` looks "wrong" without an explicit frame.

7.12 Classic SQL Interview Problems

```sql
-- 1. Second highest salary
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
-- Nth highest (N = 3)
SELECT DISTINCT salary FROM employees ORDER BY salary DESC OFFSET 2 LIMIT 1;

-- 2. Find duplicates
SELECT email, COUNT(*) FROM employees GROUP BY email HAVING COUNT(*) > 1;

-- 3. Delete duplicates, keep the lowest id
DELETE FROM employees e
USING employees e2
WHERE e.email = e2.email AND e.id > e2.id;
-- or with ROW_NUMBER():
DELETE FROM employees WHERE id IN (
  SELECT id FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) rn FROM
employees) t
  WHERE rn > 1
);

-- 4. Employees earning more than their manager
SELECT e.name FROM employees e JOIN employees m ON e.manager_id = m.id WHERE e.salary >
m.salary;

-- 5. Customers who ordered in every month of 2026 so far
SELECT customer_id FROM orders
WHERE created_at >= '2026-01-01'
GROUP BY customer_id
```

```sql
HAVING COUNT(DISTINCT DATE_TRUNC('month', created_at)) = EXTRACT(MONTH FROM now());

-- 6. Consecutive days login (gaps & islands)
WITH d AS (
  SELECT DISTINCT user_id, login_date::date AS day FROM logins
), g AS (
  SELECT user_id, day,
         day - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day))::INT AS grp
  FROM d
)
SELECT user_id, MIN(day) AS start_day, MAX(day) AS end_day, COUNT(*) AS streak
FROM g GROUP BY user_id, grp HAVING COUNT(*) >= 3;

-- 7. Cumulative % of revenue per customer (Pareto)
SELECT customer_id, total,
       SUM(total) OVER (ORDER BY total DESC) / SUM(total) OVER () AS cumulative_share
FROM (SELECT customer_id, SUM(amount) total FROM orders GROUP BY customer_id) t;

-- 8. Day-1 retention
SELECT f.first_day,
       COUNT(DISTINCT f.user_id) AS new_users,
       COUNT(DISTINCT l.user_id) AS returned_next_day
FROM (SELECT user_id, MIN(login_date) first_day FROM logins GROUP BY user_id) f
LEFT JOIN logins l ON l.user_id = f.user_id AND l.login_date = f.first_day + 1
GROUP BY f.first_day;
```

7.13 Indexes

An index is an auxiliary data structure that speeds up lookups at the cost of extra storage and slower writes.

Index type	Good for
B-Tree (default)	`=`, `<`, `>`, `BETWEEN`, `ORDER BY`, prefix `LIKE 'abc%'`
Hash	Equality only
GIN	JSONB, arrays, full-text search (`@>`, `?`, `@@`)
GiST / SP-GiST	Geometric, ranges, nearest-neighbour
BRIN	Huge, naturally ordered tables (time-series)

```sql
CREATE INDEX idx_emp_dept ON employees (department_id);
CREATE UNIQUE INDEX idx_emp_email ON employees (lower(email));       -- expression index
CREATE INDEX idx_orders_cust_date ON orders (customer_id, created_at DESC); -- composite
CREATE INDEX idx_orders_pending ON orders (created_at) WHERE status = 'pending'; -- partial
CREATE INDEX idx_cover ON orders (customer_id) INCLUDE (amount);      -- covering (index-only
scans)
CREATE INDEX CONCURRENTLY idx_x ON big_table (col);                   -- no write lock
CREATE INDEX idx_meta ON products USING GIN (metadata jsonb_path_ops);
```

Clustered vs non-clustered: a clustered index defines the physical order of rows (InnoDB primary key; only one per table). Non-clustered (secondary) indexes store keys + a pointer (or PK in InnoDB). Postgres tables are heaps; `CLUSTER` is a one-time reorder.

Composite index — leftmost prefix rule: an index on `(a, b, c)` helps queries filtering on `a`, `a,b`, `a,b,c` — but not `b` alone or `c` alone. Put equality columns first, range columns last.

When an index is NOT used:

- Function/expression on the column: `WHERE LOWER(email) = ...` (create an expression index), `WHERE created_at::date = ...`.
- Leading wildcard: `LIKE '%abc'` (use trigram GIN index `pg_trgm`).
- Implicit type casts (`varchar_col = 123`).
- Low selectivity (e.g., boolean column, most rows match) — sequential scan is cheaper.
- `OR` across different columns (sometimes; may use bitmap OR).
- Stale statistics (`ANALYZE`).

7.14 Query Optimization and EXPLAIN

```
EXPLAIN ANALYZE
SELECT o.* FROM orders o
WHERE o.customer_id = 42 AND o.created_at > now() - INTERVAL '30 days'
ORDER BY o.created_at DESC
LIMIT 20;
```

Reading the plan:

- **Seq Scan** — full table scan (fine for small tables).
- **Index Scan** — uses index, fetches rows from the heap.
- **Index Only Scan** — all needed columns in the index (best).
- **Bitmap Heap Scan** — many rows via index, read pages in order.
- Compare **estimated rows vs actual rows** — big mismatches mean stale stats.
- Look for expensive **Sort**, **Hash**, nested loops over big sets, and high `Buffers` reads.

Optimization checklist:

1. Select only needed columns (avoid `SELECT *`).
2. Index columns used in `WHERE`, `JOIN`, `ORDER BY`; use composite/covering indexes.
3. Make predicates **sargable**: `created_at >= '2026-01-01' AND created_at < '2026-02-01'` instead of `EXTRACT(MONTH FROM created_at) = 1`.
4. Replace correlated subqueries with joins/window functions when slow.
5. Prefer `EXISTS` over `IN` for large subqueries; `UNION ALL` over `UNION` when duplicates are impossible.
6. **Keyset (cursor) pagination** instead of large `OFFSET`:

```
-- OFFSET 100000 must scan and discard 100k rows
SELECT * FROM orders WHERE (created_at, id) < ($last_created_at, $last_id)
ORDER BY created_at DESC, id DESC LIMIT 20;
```

7. Batch writes; avoid N+1 queries from application code.
8. Keep statistics fresh (`ANALYZE`), vacuum (Postgres MVCC creates dead tuples).
9. Partition very large tables; consider materialized views for heavy reports.

7.15 Transactions and ACID

- **Atomicity** — all or nothing.
- **Consistency** — constraints always hold.
- **Isolation** — concurrent transactions don't interfere (to the configured level).
- **Durability** — committed data survives crashes (write-ahead log).

```sql
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
SAVEPOINT before_credit;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
-- ROLLBACK TO SAVEPOINT before_credit;
COMMIT;
```

Isolation Levels and Anomalies

Level	Dirty read	Non-repeatable read	Phantom read	Notes
Read Uncommitted	Possible	Possible	Possible	Postgres treats as Read Committed
Read Committed	No	Possible	Possible	Postgres default
Repeatable Read	No	No	Possible (No in PG)	MySQL InnoDB default
Serializable	No	No	No	May abort with serialization errors → retry

- **Dirty read:** reading uncommitted data of another transaction.
- **Non-repeatable read:** same row read twice returns different values.
- **Phantom read:** same query returns new rows inserted by another transaction.
- **Lost update:** two transactions read-modify-write the same row; one overwrites the other. Prevent with `UPDATE ... SET x = x - 1`, `SELECT ... FOR UPDATE`, or optimistic locking.

```sql
-- Pessimistic locking
BEGIN;
SELECT stock FROM products WHERE id = 7 FOR UPDATE;      -- others wait
UPDATE products SET stock = stock - 1 WHERE id = 7;
COMMIT;

-- Job queue pattern: workers grab different rows without blocking
SELECT id FROM jobs WHERE status = 'queued'
ORDER BY id LIMIT 10 FOR UPDATE SKIP LOCKED;

-- Optimistic locking
UPDATE products SET stock = 9, version = version + 1
WHERE id = 7 AND version = 3;          -- 0 rows updated → someone else changed it → retry
```

Deadlocks occur when two transactions wait on each other's locks; the DB aborts one. Avoid by locking rows in a consistent order and keeping transactions short.

MVCC (Postgres, InnoDB): readers don't block writers and writers don't block readers; each transaction sees a snapshot.

7.16 Normalization

Normal form	Rule
1NF	Atomic values; no repeating groups/arrays in a column; each row unique
2NF	1NF + no partial dependency (non-key column depends on whole composite key)
3NF	2NF + no transitive dependency (non-key → non-key)
BCNF	Every determinant is a candidate key

Example: `orders(order_id, customer_id, customer_name, product_id, product_name, qty)` violates 2NF/3NF → split into `customers`, `products`, `orders`, `order_items`.

Denormalization (duplicating data, summary tables, materialized views) trades write complexity for read speed — common in reporting/analytics (OLAP) systems.

Relationships: one-to-one (FK + UNIQUE), one-to-many (FK on the "many" side), many-to-many (junction table with two FKs and a composite PK/unique).

7.17 Views, Materialized Views, Functions, Triggers

```
CREATE VIEW active_employees AS
SELECT id, name, department_id FROM employees WHERE is_active;

CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT DATE_TRUNC('month', created_at) AS month, SUM(amount) AS revenue
FROM orders WHERE status = 'paid' GROUP BY 1;
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue;  -- needs a unique index

-- Function (PL/pgSQL)
CREATE OR REPLACE FUNCTION dept_headcount(p_dept INT) RETURNS INT AS $$
BEGIN
  RETURN (SELECT COUNT(*) FROM employees WHERE department_id = p_dept);
END;
$$ LANGUAGE plpgsql STABLE;

-- Trigger: maintain updated_at
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at := now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_orders_updated
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- Stored procedure (can manage transactions)
CREATE PROCEDURE transfer(src INT, dst INT, amt NUMERIC) LANGUAGE plpgsql AS $$
BEGIN
  UPDATE accounts SET balance = balance - amt WHERE id = src;
  UPDATE accounts SET balance = balance + amt WHERE id = dst;
  COMMIT;
END; $$;
CALL transfer(1, 2, 100);
```

7.18 Working with JSON, Strings and Dates (PostgreSQL)

```sql
-- JSONB
SELECT metadata->>'color' AS color,              -- text
       metadata->'dims'->>'w' AS width,
       metadata #>> '{dims,h}' AS height
FROM products
WHERE metadata @> '{"brand": "acme"}'            -- containment (GIN-indexable)
  AND metadata ? 'warranty';                     -- key exists

UPDATE products SET metadata = jsonb_set(metadata, '{color}', '"red"');
SELECT jsonb_agg(row_to_json(e)) FROM employees e;          -- rows → JSON array
SELECT * FROM jsonb_to_recordset('[{"a":1},{"a":2}]') AS x(a INT);

-- Strings
SELECT UPPER(name), LENGTH(name), SUBSTRING(name FROM 1 FOR 3), TRIM(name),
       CONCAT(first, ' ', last), SPLIT_PART(email, '@', 2), REPLACE(name, 'a', 'A'),
       REGEXP_REPLACE(phone, '\D', '', 'g')
FROM employees;

-- Dates
SELECT now(), CURRENT_DATE, now() - INTERVAL '7 days',
       DATE_TRUNC('week', created_at), EXTRACT(DOW FROM created_at),
       AGE(now(), hired_at), TO_CHAR(created_at, 'YYYY-MM-DD HH24:MI')
FROM orders;

-- Full-text search
SELECT * FROM articles
WHERE to_tsvector('english', title || ' ' || body) @@ plainto_tsquery('english', 'django orm');
```

7.19 Security, Roles and SQL Injection

```sql
CREATE ROLE app_readonly LOGIN PASSWORD '...';
GRANT CONNECT ON DATABASE shop TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;
REVOKE DELETE ON orders FROM app_user;

-- Row-Level Security (multi-tenant)
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.tenant_id')::INT);
```

SQL injection: never build SQL with string concatenation of user input.

```python
# BAD
cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")   # email = "' OR 1=1 --"
# GOOD (parameterized)
cursor.execute("SELECT * FROM users WHERE email = %s", [email])
```

```javascript
// BAD:  pool.query(`SELECT * FROM users WHERE id = ${req.params.id}`)
// GOOD:
pool.query('SELECT * FROM users WHERE id = $1', [req.params.id]);
```

7.20 Scaling Databases

- **Vertical scaling** (bigger machine) vs **horizontal** (more machines).
- **Read replicas** — offload reads; beware replication lag (read-your-writes).
- **Connection pooling** — PgBouncer / RDS Proxy; DB connections are expensive.

- **Partitioning** (single DB, split table):

```
CREATE TABLE events (id BIGINT, created_at TIMESTAMPTZ, payload JSONB)
PARTITION BY RANGE (created_at);
CREATE TABLE events_2026_10 PARTITION OF events
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
-- Partition pruning: queries filtering on created_at only scan relevant partitions.
```

- **Sharding** (split across DBs by shard key, e.g., `tenant_id`) — complex cross-shard joins/transactions.
- **CAP theorem:** in a network partition choose Consistency or Availability. **OLTP** (transactions, normalized) vs **OLAP** (analytics, columnar, denormalized).
- **SQL vs NoSQL:** SQL — schema, joins, ACID, complex queries; NoSQL (MongoDB, DynamoDB, Redis, Cassandra) — flexible schema, horizontal scale, access-pattern-driven design, often eventual consistency.

7.21 Quick Reference — Postgres vs MySQL

Feature	PostgreSQL	MySQL (InnoDB)
Auto-increment	`SERIAL` / `GENERATED ALWAYS AS IDENTITY`	`AUTO_INCREMENT`
Upsert	`ON CONFLICT ... DO UPDATE`	`ON DUPLICATE KEY UPDATE`
Returning rows	`RETURNING`	Not supported (use `LAST_INSERT_ID()`)
String aggregate	`STRING_AGG`	`GROUP_CONCAT`
Case-insensitive LIKE	`ILIKE`	Depends on collation (default CI)
JSON	`JSONB` with GIN indexes	`JSON` with generated-column indexes
Default isolation	Read Committed	Repeatable Read
Full outer join	Yes	No (emulate with UNION)